Omni Pendulum Robot

Balancing a 2-DOF inverted pendulum on a mecanum-wheeled AgileX LIMO with ROS 2 Jazzy and Gazebo Harmonic on Ubuntu 24.04

Project review guide, written to be read from scratch: concepts, commands, robot model, control strategy, test results and a line-by-line walkthrough of the code.

Repository: `github.com/Surya-ganesh5353/omni_pendulum_ws`
Package: `omni_pendulum_sim`

Contents

1. The project at a glance
2. Foundations from scratch: ROS 2, Gazebo, the bridge, inverted pendulums, mecanum wheels
3. Build and run it, step by step: every command
4. The robot and world model
5. What happens behind the scenes
6. The control strategy
7. Test results and known limits
8. How the project developed: problems found, decisions made, next steps
9. Line-by-line code walkthrough
    1. balance_controller.py
    2. disturbance.py
    3. keyboard_nudge.py
    4. sim.launch.py
    5. robot.urdf.xacro
    6. balance_world.sdf
    7. setup.py
    8. package.xml

Part 1: The project at a glance

1.1 The goal

A pole standing upright on a moving robot wants to fall over. This project makes the robot keep it up, the way you can balance a broomstick on your palm by moving your hand under it. Because the robot has mecanum wheels, it can move in any direction, so the pole can fall in any direction and still be caught.

The work so far is a simulation. The next step is running it on a real LIMO in a motion-capture (mocap) room.

1.2 What you see when it runs

1. Gazebo opens with an AgileX LIMO carrying a yellow platform. A 0.5 m red rod with a blue ball on top stands on a free two-axis hinge on the platform.
2. About 4 s in, one tiny 0.1 N nudge starts the pole falling. The robot does not react yet, so you see the fall.
3. At 8° of tilt the robot makes its first catch: it drives toward the fall to get back under the pole.
4. From then on it repeats a cycle roughly once a second: the robot lets go, gravity tips the pole, and at about 6° the robot catches it again.
5. You can push the pole at any time with the arrow keys. A push makes the robot react immediately.

1.3 Key numbers

Item	Value	Item	Value
Pole	0.5 m rod, 0.1 kg, plus 0.1 kg ball	Control rate	200 Hz (every 5 ms)
Physics step	1 ms	Time for a tilt to grow by a factor of e	0.21 s
Cycle period	about 0.95 s	Peak tilt in the cycle	about 6.7°
Speed in the cycle	0.27 m/s average, 0.72 m/s peak	Speed limit	2 m/s
Strongest push survived in all directions	1.2 N at the top of the pole	Floor space in 120 s (one run)	0.74 m from start

1.4 The files

File	What it does
`omni_pendulum_sim/balance_controller.py`	The controller: reads the pole's angles, commands the base's velocity
`omni_pendulum_sim/disturbance.py`	Simulation only: the start-up nudge and manual pushes
`omni_pendulum_sim/keyboard_nudge.py`	Optional terminal arrow-key input
`launch/sim.launch.py`	Starts everything with one command
`urdf/robot.urdf.xacro`	The robot model and its Gazebo plugins
`worlds/balance_world.sdf`	The simulated world
`config/gui.config`	Gazebo window layout, closer camera, key-press plugin
`meshes/limo/*_lite.dae`	Simplified LIMO 3D shapes
`setup.py`, `setup.cfg`, `package.xml`, `resource/`	ROS 2 package build files
`test/`	Code style tests (flake8, pep257)

Part 2: Foundations from scratch

This part explains every concept the rest of the guide relies on. If you already know ROS 2 and Gazebo, skip to Part 3.

2.1 ROS 2: how robot programs talk

ROS 2 (Robot Operating System 2) is not really an operating system. It is a set of libraries and tools that lets separate programs on a robot exchange data. This project uses the **Jazzy** release, which is the one built for Ubuntu 24.04.

Concept	Meaning	In this project
Node	One running program with one job	`balance_controller`, `disturbance`
Topic	A named channel. Anyone can publish to it or subscribe to it, without knowing who is on the other end	`/cmd_vel`, `/joint_states`
Message	The typed data sent on a topic	`geometry_msgs/Twist` holds linear and angular velocity
Publisher / subscriber	A node's sending and receiving ends. A subscriber runs a *callback* function for each message	The controller's `joint_callback`
Parameter	A setting of a node that can be changed without editing code	`catch_tilt_rad`
Package	A folder of code and files that gets built and installed as a unit	`omni_pendulum_sim`
Workspace	A folder containing packages (`src/`) and their build results	`~/omni_pendulum_ws`
colcon	The build tool. It creates `build/`, `install/` and `log/`	`colcon build`
Sourcing	Running a `setup.bash` file so the current terminal can find ROS and your packages. Every new terminal needs it	`source install/setup.bash`
Launch file	A Python file that starts many programs at once	`sim.launch.py`
TF	A system tracking where every part of the robot is relative to the others	Published by `robot_state_publisher`
Simulation time	Nodes read the clock from the simulator instead of the computer, using `use_sim_time`	`/clock`

A minimal node looks like this. The project's nodes follow the same pattern:

```
class MyNode(Node):
    def __init__(self):
        super().__init__('my_node')                                  # name on the ROS network
        self.create_subscription(JointState, '/joint_states', self.cb, 10)  # 10 = queue size
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)

    def cb(self, msg):                # runs for every message received
        self.pub.publish(Twist())     # publishes a zero velocity

rclpy.init(); rclpy.spin(MyNode())    # spin: wait for messages and call the callbacks
```

2.2 Gazebo: the simulated world

Gazebo Harmonic is a robot simulator: it computes gravity, contacts and friction, and draws the result. ROS 2 Jazzy is paired with it. Older tutorials use “Gazebo Classic”, whose plugins do *not* work here, which matters later when importing the LIMO.

Concept	Meaning
World	The environment: physics settings, light, ground, and the robots in it
Model	One object in the world, such as our robot
Link	A rigid body. It has *inertial* properties (mass), a *visual* shape (what you see) and a *collision* shape (what the physics touches)
Joint	A connection between links: `fixed` (rigid), `revolute` (hinge with limits) or `continuous` (hinge without limits, like an axle)
System (plugin)	Code added to the simulation. Examples: MecanumDrive turns velocity commands into wheel speeds; JointStatePublisher reports joint angles
Server and GUI	The server computes physics; the GUI is the window. They can run together or separately
Gazebo Transport	Gazebo's own message system. It is similar to ROS topics but separate from them
Real-time factor	Simulated seconds per real second. Below 1 means the computer can't keep up, and the simulation runs in slow motion

Three robot and world file formats

Format	What it is	Used for
URDF	ROS's XML description of a robot: links and joints	The robot
xacro	URDF plus variables, maths and macros. It expands into plain URDF	`robot.urdf.xacro`
SDF	Gazebo's native format, which can also describe whole worlds. Gazebo converts URDF to SDF when spawning	`balance_world.sdf`

2.3 The bridge: connecting ROS 2 and Gazebo

ROS 2 and Gazebo use different message systems, so a topic in one is invisible to the other. `ros_gz_bridge` translates selected topics. Each is declared in the launch file like this:

```
/cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist
 topic   ROS message type      | Gazebo message type

 ]  means ROS to Gazebo
 [  means Gazebo to ROS
```

2.4 Why an inverted pendulum falls, and how it is caught

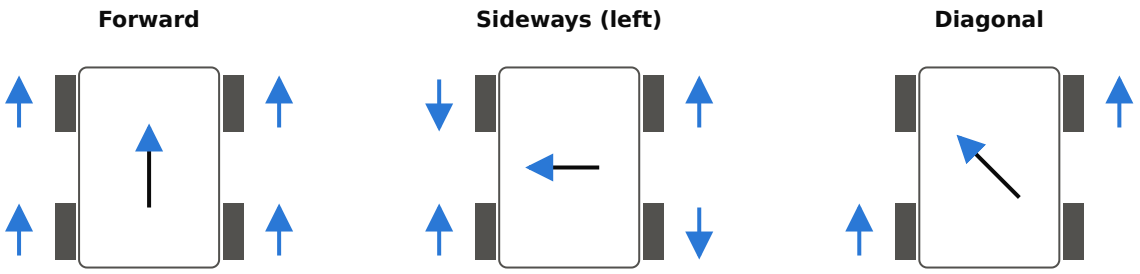
A pole balanced upright is in an **unstable equilibrium**. At exactly 0° gravity has no sideways effect, but any tiny tilt lets gravity pull it further, which increases the tilt, and so on. The tilt grows *exponentially*.

To catch it, the base must **accelerate** toward the fall. An acceleration under the pole's pivot produces a turning effect on the pole that opposes gravity's. Moving at a constant speed does nothing; the pole only feels acceleration, just as you only feel pushed back in a car while it speeds up. This is the most important idea in the controller.

It also explains the counter-intuitive part: to **stop**, the robot must first let the pole lean slightly backwards, or the deceleration would throw the pole forward.

2.5 How mecanum wheels move sideways

A mecanum wheel has small angled rollers around its rim, so it pushes the robot along a diagonal instead of straight ahead. With diagonals arranged in an X, combinations of wheel speeds produce motion in any direction:



Top view, robot front at the top. Blue arrows: each wheel's spin direction. Black arrow: the resulting robot motion. FL, FR, RL, RR = front-left, front-right, rear-left, rear-right.

The exact rule, for wheel radius r and k = half of (wheelbase + wheel separation):

$$\mathrm{FL} = (v_x - v_y - k\omega)/r \quad \mathrm{FR} = (v_x + v_y + k\omega)/r$$
$$\mathrm{RL} = (v_x + v_y - k\omega)/r \quad \mathrm{RR} = (v_x - v_y + k\omega)/r$$

On a pure diagonal ($v_x = v_y$), FL and RR are zero, so only two wheels do the work. This is why the diagonal turned out to be the robot's weakest direction in testing.

Part 3: Build and run it, step by step

3.1 Install the software (once)

Follow the official instructions at `docs.ros.org/en/jazzy/Installation.html` to install ROS 2 Jazzy. Then install what this project needs:

```
sudo apt install ros-jazzy-ros-gz ros-jazzy-xacro ros-jazzy-robot-state-publisher \
                 python3-colcon-common-extensions git
```

3.2 Get the code

```
git clone https://github.com/Surya-ganesh5353/omni_pendulum_ws.git ~/omni_pendulum_ws
```

The repository is a complete workspace: the package is already in `src/`.

3.3 Build

```
source /opt/ros/jazzy/setup.bash          # make ROS available in this terminal
cd ~/omni_pendulum_ws
colcon build --symlink-install            # build the workspace
source install/setup.bash                 # make this package available too
```

`--symlink-install` links to your source files instead of copying them, so edits to Python, launch, URDF, world and config files apply at the next launch without rebuilding. Rebuild only after *adding* a new file or program.

3.4 Run

```
ros2 launch omni_pendulum_sim sim.launch.py
```

Launch option	Effect
`gui:=false`	No Gazebo window (used for automated tests)
`balance:=false`	No controller, so you can watch the pole fall
`push:=false`	No disturbance node: no start-up nudge and no manual nudges, so the pole stays upright

Stop with **Ctrl+C** in the launch terminal. Closing only the window can leave processes behind.

3.5 Push the pole

Method	How
Gazebo window	Click the 3D view, then use the arrow keys. ↑/↓ = forward/back, ←/→ = left/right, `+` / `-` = stronger/weaker (starts at 0.8 N)
Terminal	`ros2 run omni_pendulum_sim keyboard_nudge` (same keys, `q` quits)
Exact force	`ros2 topic pub --once /nudge geometry_msgs/msg/Vector3 "{x: 0.5, y: 0.0}"`

3.6 Inspect a running system

This is real output from this project, captured during a test run:

```
$ ros2 node list
/balance_controller
/disturbance
/robot_state_publisher
/ros_gz_bridge

$ ros2 topic list
/clock                 /keyboard/keypress     /robot_description     /tf_static
/cmd_vel               /nudge                 /rosout
/world/balance_world/wrench/clear
/cmd_vel_ref           /odom                  /tf
/world/balance_world/wrench/persistent
/ground_truth/odom     /parameter_events      /joint_states

$ ros2 topic info /cmd_vel
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1

$ ros2 param get /balance_controller catch_tilt_rad
Double value is: 0.1

$ gz model --list
Available models:
    - ground_plane
    - omni_pendulum
```

(The topic list is arranged in columns here to save space.) Other useful commands:

Command	Shows
`ros2 topic echo /cmd_vel`	Live velocity commands
`ros2 topic hz /joint_states`	How often messages arrive
`ros2 param list /balance_controller`	All controller settings
`gz topic -l`	Topics on the Gazebo side
`colcon test --packages-select omni_pendulum_sim`, then `colcon test-result --verbose`	Runs the code style tests

3.7 Change a setting

Parameters are read when a node starts. To try one without editing code, launch without the controller and start it yourself in a second terminal:

```
ros2 launch omni_pendulum_sim sim.launch.py balance:=false
ros2 run omni_pendulum_sim balance_controller --ros-args -p use_sim_time:=true -p
catch_tilt_rad:=0.08
```

Controller parameter	Default	Effect
`k_theta`, `k_omega`, `k_pos`, `k_vel`	33.4, 7.1, 1.3, 3.1	Control gains (Part 6)
`engage_tilt_rad`	0.14 (8°)	Free fall before the first catch
`catch_tilt_rad`	0.1 (5.7°)	Tilt that triggers a catch. Larger: bigger swings, less margin
`catch_rate`	0.6 rad/s	Catch at once if the pole moves faster than this
`release_tilt_rad`, `release_rate`	0.015, 0.1 rad/s	How settled the pole must be before letting go
`max_lean_rad`	0.06 (3.4°)	Most the controller may lean the pole to brake. Keep at 0.05 or more
`max_return_speed`	0.3 m/s	Speed limit of the drive home
`max_accel`	6.0 m/s^2	Must match what the wheels can deliver
`max_vel`	2.0 m/s	Absolute speed limit
`max_tilt_rad`	0.6 (34°)	Give up and stop beyond this tilt
`control_period`	0.005 s	Time between control steps

Disturbance parameter	Default	Effect
`release_delay`, `release_force`	4 s, 0.1 N	The single start-up nudge
`key_force`, `key_force_step`	0.8 N, 0.2 N	Arrow-key push strength
`nudge_duration`	0.15 s	Length of a push
`min_contact_height`, `max_contact_height`	0.3 m, 0.5 m	Where on the rod a push lands (chosen at random)

3.8 Save your work to GitHub

```
git status                        # what changed
git add README.md src docs        # choose what to save
git commit -m "Describe the change"
git push
```

3.9 Troubleshooting

Symptom	Cause and fix
`Package 'omni_pendulum_sim' not found`	This terminal isn't sourced: `source ~/omni_pendulum_ws/install/setup.bash`
Robot missing, or “Moved backwards in time”	A leftover Gazebo is running: `pkill -f "gz sim"`, then relaunch
`XML parsing error` from xacro	The model file is damaged, often by text pasted into it or an old VS Code tab saved over it. Compare with `git diff`
Arrow keys do nothing in Gazebo	Click the 3D view first, or use `keyboard_nudge`
Yellow QML “Binding loop” warnings	Harmless; they come from Gazebo's own GUI files
Gazebo shows “not responding”	Wait a moment. On laptops with two graphics cards, try running Gazebo on the dedicated one
Pole falls after a strong push	Expected above about 1.2–1.5 N at the top of the pole

Part 4: The robot and world model

4.1 Structure of the robot

```
base_footprint          (empty link at ground level; the model's root)
 └─ base_joint (fixed)
    base_link           LIMO chassis, 2.16 kg
     ├─ 4 × wheel_joint (continuous)
     │     wheel_link   0.5 kg each, radius 45 mm
     └─ platform_joint (fixed)
        platform_link   yellow plate on 4 posts, top at 0.266 m
         └─ pendulum_mount_joint (fixed)
            pendulum_mount      small puck; the pivot sits on top, at 0.278 m
             └─ pendulum_pitch_joint (revolute, about Y: tips toward +X)
                dummy_pitch_link     1 g link between the two hinges
                 └─ pendulum_roll_joint (revolute, about X: tips toward -Y)
                    pendulum_link    rod + ball, 0.2 kg
```

Two perpendicular hinges in series form a **universal joint**, so the pole can tip in any direction. Their damping is almost zero, so nothing holds the pole up except the robot's motion. Each hinge is limited to ±90°.

4.2 Where the LIMO came from

The shapes and dimensions come from AgileX's `limo_ros2` repository, with three adaptations:

- **No mecanum model exists there.** It has only diff-drive and Ackermann versions, for Gazebo Classic. Their plugins don't exist in Gazebo Harmonic, so the drive was rebuilt with Harmonic's MecanumDrive system.
- **The meshes were too heavy.** The body had 755,790 triangles in 1,619 parts, and each wheel 140,290, which froze Gazebo. A one-off script merged the parts by colour and reduced the detail: the body to 41,255 triangles and the wheel to 14,499. Total size went from 63.6 MB to 3.8 MB.
- **The top isn't flat.** Measuring the mesh showed a dip at 0.146 m and a sloped cover peaking at 0.254 m, so the pole's mount stands on a platform just above the peak.

4.3 Simulating mecanum wheels

Modelling dozens of tiny rollers would be slow and numerically unstable. Each wheel is instead a plain cylinder with **anisotropic friction**, meaning it differs by direction:

```
<mu>1.0</mu>     full grip along the diagonal fdir1
<mu2>0.0</mu2>   no grip across it
<fdir1 gz:expressed_in="base_footprint">1 -1 0</fdir1>
```

That is the net effect of real rollers. The direction is expressed in `base_footprint` so it doesn't spin with the wheel. It uses `base_footprint` rather than `base_link` because, when URDF is converted to SDF, links joined by fixed joints are merged into their parent, so `base_link` stops being a separate link.

4.4 The Gazebo plugins in the model

Plugin	Job
MecanumDrive	Converts `/cmd_vel` into four wheel speeds; publishes wheel-based `/odom`
JointStatePublisher	Publishes all joint angles and rates on `/joint_states`. This is the controller's only sensor
OdometryPublisher	Publishes the robot's true motion on `/ground_truth/odom`, for testing. Wheel odometry can't detect wheel slip; this can

The world adds Physics, UserCommands (spawning), SceneBroadcaster (drawing) and **ApplyLinkWrench** (pushing links). `gui.config` adds Key Publisher to the window, and moves the camera from 6 m to about 1.6 m away, looking along +X, so screen-up is robot-forward.

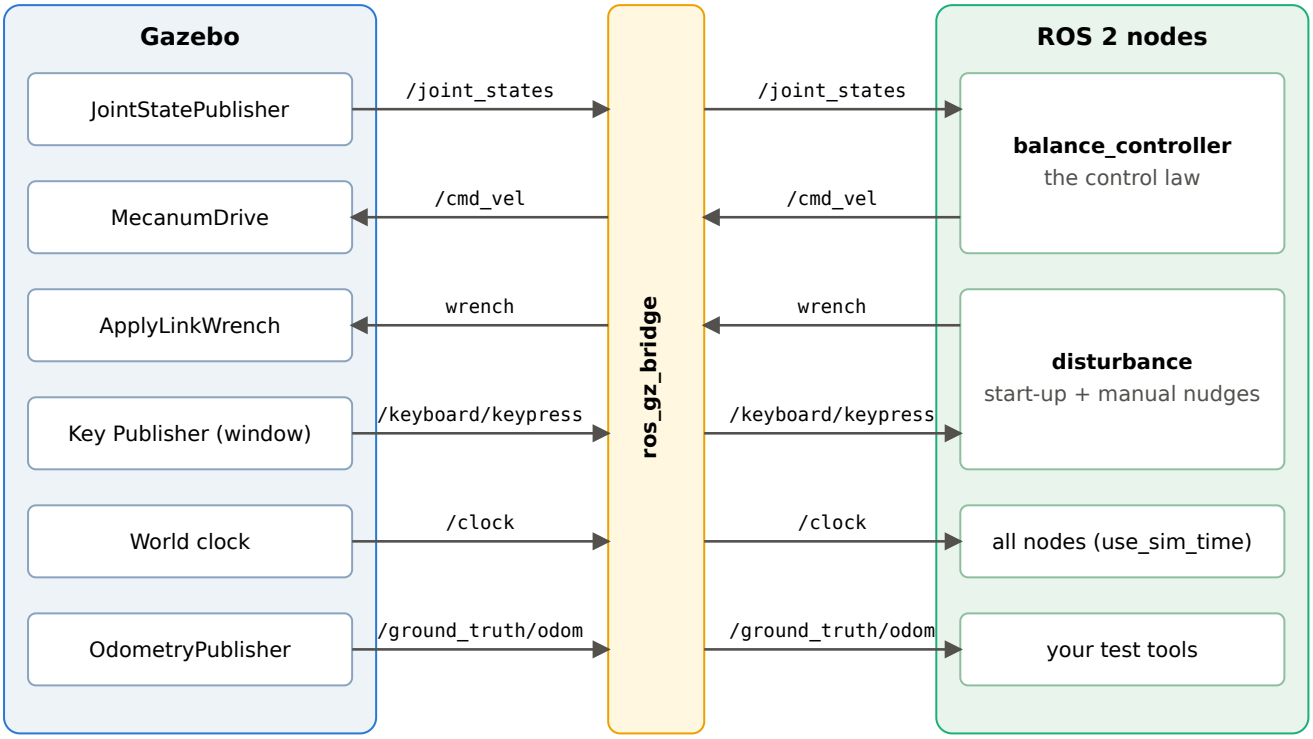
4.5 Axis conventions

Quantity	Positive direction
Robot X / Y	Forward / left
Pitch angle	Pole top leans forward (+X)
Roll angle	Pole top leans **right** (−Y), by the right-hand rule about +X

Because positive roll points the opposite way to +Y, the Y controller works in negated coordinates. That explains the minus signs in the control loop.

Part 5: What happens behind the scenes

5.1 System diagram



Two ROS-side inputs reach `disturbance` without passing through Gazebo: `/nudge` and key presses from `keyboard_nudge`. `robot_state_publisher` also runs, publishing the model and transforms.

5.2 Topics

Topic	Message type	From → to
`/joint_states`	sensor_msgs/JointState	Gazebo → controller, robot_state_publisher
`/cmd_vel`	geometry_msgs/Twist	controller → MecanumDrive
`/cmd_vel_ref`	geometry_msgs/Twist	optional driving command → controller
`/world/balance_world/wrench/persistent`	ros_gz_interfaces/EntityWrench	disturbance → ApplyLinkWrench
`/world/balance_world/wrench/clear`	ros_gz_interfaces/Entity	disturbance → ApplyLinkWrench
`/keyboard/keypress`	std_msgs/Int32	Key Publisher, keyboard_nudge → disturbance
`/nudge`	geometry_msgs/Vector3	you → disturbance
`/odom`	nav_msgs/Odometry	MecanumDrive → ROS (wheel-based)
`/ground_truth/odom`	nav_msgs/Odometry	OdometryPublisher → ROS (true motion)
`/clock`	rosgraph_msgs/Clock	Gazebo → all nodes

5.3 What happens when you launch

1. `ros2 launch` runs `generate_launch_description()`, which returns the list of things to start.
2. `xacro` expands the model file into plain URDF: variables are substituted, maths evaluated, macros expanded.
3. `robot_state_publisher` starts and publishes that text on `/robot_description`.
4. Gazebo starts with the world file and window layout, and loads the world's systems.
5. The bridge starts and links the listed topics.
6. The controller and disturbance nodes start and wait for messages.
7. After 2 s, `ros_gz_sim create` reads `/robot_description` and asks Gazebo to spawn the robot 5 mm above the ground. Gazebo converts the URDF to SDF and starts the model's plugins.
8. About 4 s after the disturbance node's clock starts, it sends the release nudge.

5.4 One control step, from physics to wheels

1. Gazebo advances physics by 1 ms: gravity, contacts, friction, joint motion.
2. JointStatePublisher sends the joint angles and rates. The bridge converts them to a `JointState` message.
3. `joint_callback` runs. If under 5 ms of simulated time has passed since the last control step, it ignores the message.
4. Otherwise it computes the command (Part 6) and publishes a `Twist` on `/cmd_vel`.
5. The bridge passes it to MecanumDrive, which computes the four wheel speeds.
6. The wheels' friction pushes on the ground, the robot moves, and that acceleration acts on the pole.

5.5 Simulation time

A loaded computer may run Gazebo below real time. All nodes use `use_sim_time`, and the controller takes its time step from message timestamps, so the maths stays consistent with the physics whatever the computer's speed. On a real robot this setting must be off.

5.6 How a nudge is applied

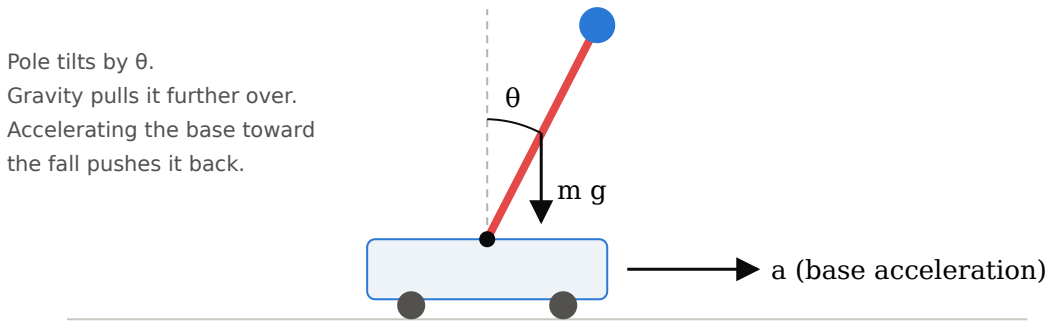
A person pushes the rod at some height, but ApplyLinkWrench acts at the link's origin, which is the pivot, and a force through the pivot can't tip the pole. So a force F at height h is applied as its equivalent torque, $\tau = F \times h$. Its strength follows a half-sine over 0.15 s: smooth like a finger, not a sudden jolt.

Two Gazebo details shaped the code:

- **Persistent wrenches add up** instead of replacing each other, so the node tracks the current total and sends only the *change* every 10 ms.
- **They stay stored** until cleared, so the node sends a clear 0.1 s after each nudge. The delay is needed because updates and clears travel on separate topics and could arrive out of order.

Part 6: The control strategy

6.1 The physics of the pole



Taking moments about the pivot, with J = moment of inertia about the pivot, m = pendulum mass, l = pivot to centre of mass:

$$J \cdot \ddot{\theta} = m\,g\,l \cdot \sin\theta - m\,l \cdot a \cdot \cos\theta$$

For small angles $\sin\theta \approx \theta$ and $\cos\theta \approx 1$, so:

$$\ddot{\theta} = \alpha\,\theta - \beta\,a \qquad \text{where } \alpha = m\,g\,l\,/\,J, \quad \beta = m\,l\,/\,J$$

Quantity	Value	How it is obtained or what it means
J	0.0334 kg·m^2	Rod: $m_r L^2/3$, plus ball: $m_b L^2 + 2m_b r^2/5$
m l	0.075 kg·m	0.1 × 0.25 + 0.1 × 0.5
α	22.06 s^{-2}	Gravity's strength on the tilt
β	2.25 m^{-1}	How strongly base acceleration acts on the tilt
$1/\sqrt{\alpha}$	0.213 s	A free-falling tilt grows by a factor of e in this time
$\alpha/\beta = g$	9.81 m/s^2	Holding a steady lean θ requires $a \approx g\theta$

6.2 Why “tilt to velocity” cannot work

The first draft commanded a velocity proportional to tilt and tilt rate, $v = K_p\theta + K_d\dot{\theta}$. Since acceleration is the derivative of velocity, $a = K_p\dot{\theta} + K_d\ddot{\theta}$. Substituting into the model gives:

$$(1 + \beta K_d)\,\ddot{\theta} + \beta K_p\,\theta - \alpha\,\theta = 0$$

The last term's coefficient is negative whatever the gains, which always leaves an unstable root. So the design must make acceleration depend on θ directly: with $a = k\theta$ the model becomes $\ddot{\theta} = (\alpha - \beta k)\theta$, which is restoring when $k > \alpha/\beta = 9.81$. The controller computes an acceleration and **integrates it into the velocity command**, since the base accepts only velocity:

$$v \leftarrow v + a\cdot\Delta t \qquad p \leftarrow p + v\cdot\Delta t$$

6.3 One axis at a time

For small tilts, pitch responds only to X acceleration and roll only to Y acceleration, and a mecanum base can accelerate along X and Y independently. So the 2D problem splits into two identical 1D cart-pole problems, with one `AxisController` for each.

6.4 Finding the gains with LQR

Each axis has four state variables: tilt, tilt rate, base position and base velocity:

$$\dot{\theta} = \alpha\theta - \beta a, \quad \dot{p} = v, \quad \dot{v} = a$$

LQR (Linear Quadratic Regulator) finds the gains in $a = k_\theta\theta + k_\omega\dot{\theta} + k_p p + k_v v$ that minimise a cost combining state error (weights Q) and acceleration used (weight R):

$$J_{cost} = \int (x^T Q x + R a^2) \, dt$$

It was solved with `scipy.linalg.solve_continuous_are`, using Q = diag(20, 1, 0.5, 1) and R = 0.3:

k_θ	k_ω	k_p	k_v	Closed-loop poles
33.4	7.1	1.3	3.1	−6.5, −4.49, −0.98 ± 0.09j

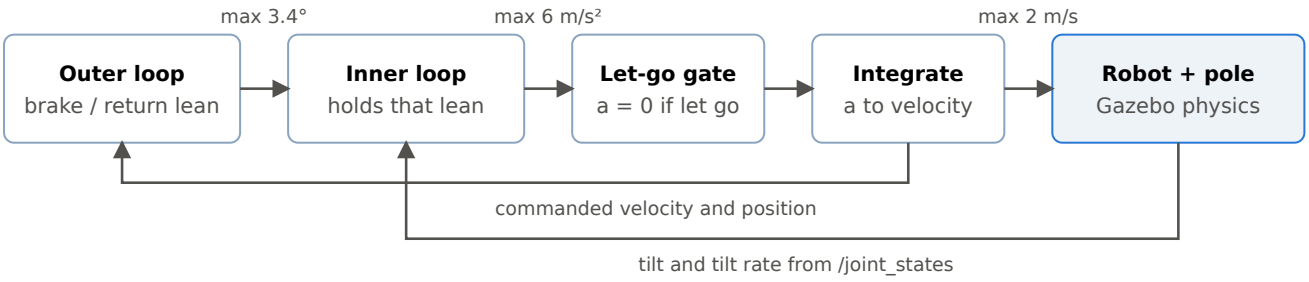
All poles are in the left half-plane, so the linear model is stable. The fast pair handles the pole; the slow pair brings the robot home. These gains are deliberately soft: an earlier, stiffer design (k_θ = 60) proved no better at recovering from nudges. The fast poles can't go far below $\sqrt{\alpha} \approx 4.7\ s^{-1}$, because the controller has to be at least as quick as the pole falls.

6.5 The cascade: limiting braking without limiting balance

The same law can be rewritten exactly as two levels, where e_p and e_v are the position and velocity errors:

$$lean_{ref} = -(k_p\, e_p + k_v\, e_v) / k_\theta \qquad a = k_\theta(\theta - lean_{ref}) + k_\omega\theta$$

Multiplying out returns the original four-term law. The benefit is meaning: the outer level says *how far to lean the pole to slow down or go home*, and the inner level *keeps the pole at that lean*. Now each can be limited for a physical reason.



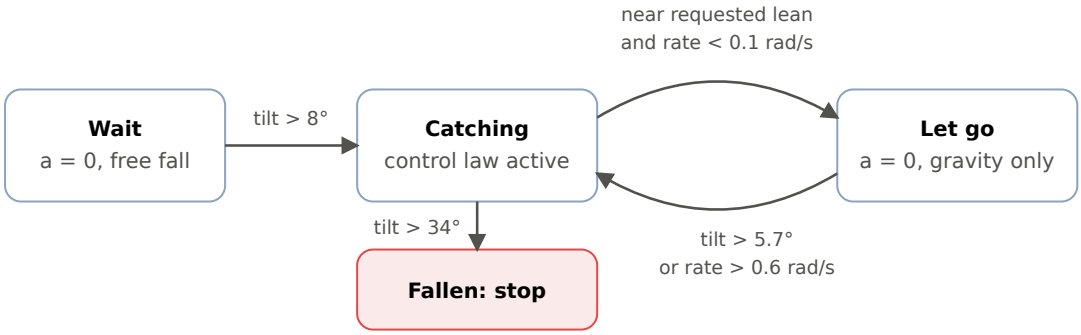
Limit	Value	Why
Lean request	3.4°	Braking can't throw the pole far over to the other side. This fixed the overshoot. A steady 3.4° lean allows about g × 0.06 = 0.59 m/s^2 of braking, so stops are gentle
Position term	0.3 m/s	However far a push carried the robot, it drives home calmly
Acceleration	6 m/s^2	Never ask for more than the wheels can deliver (below)
Speed	2 m/s	Absolute limit. The limited value is written back to the controller's state, so its internal velocity can't drift away from the command (wind-up)

The limits on acceleration and speed apply to the 2D vector, not per axis. Per-axis clipping would bend a diagonal command toward one axis.

> **A key assumption.** The controller has no velocity sensor: it uses its own commanded velocity as its estimate. That is valid only if the base really follows the command, which is why `max_accel` must match the wheels. When it was 8 m/s^2, testing showed the real robot at 1.08 m/s when commanded 2.0 m/s, so the controller braked too late and dropped the pole. **This matters on a real robot.**

6.6 The let-go / catch cycle

A good controller holds a pole almost perfectly still, under 0.1° here, which looks frozen and shows no gravity at work. So balancing runs as a cycle in which the controller repeatedly stops controlling:



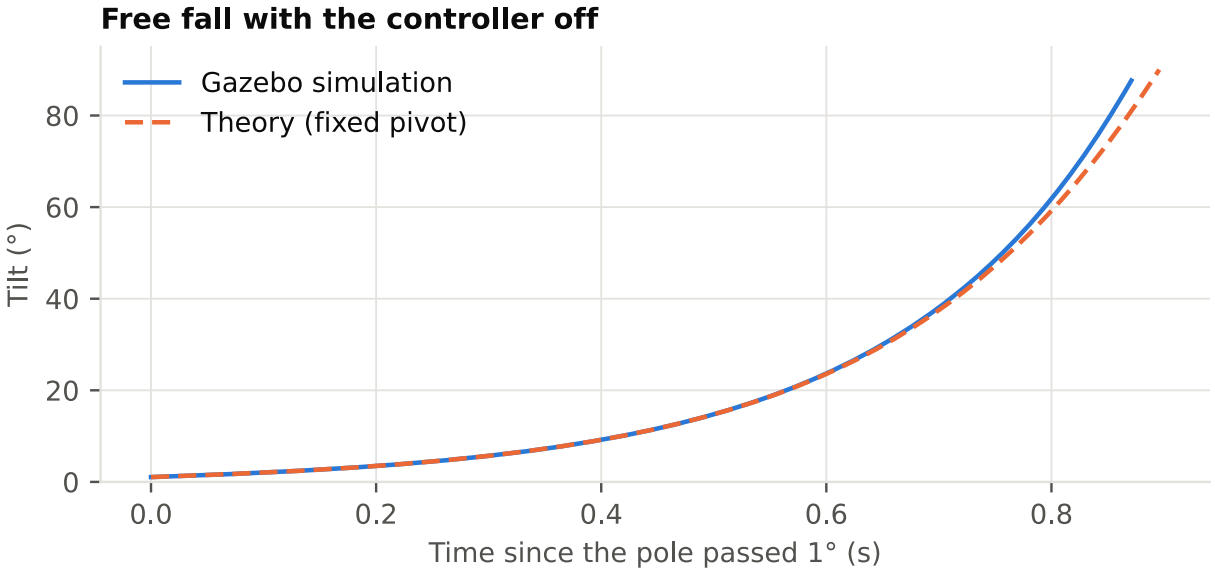
- **While letting go,** acceleration is zero and the robot keeps its current velocity. The only thing moving the pole is gravity.
- **Catch at 5.7°:** this angle sets the size of the swings. 10° was too close to the limit of what can be caught.
- **Or catch on fast motion:** under gravity alone the pole reaches only $\sqrt{\alpha\theta} = 4.7 \times 0.1 = 0.47$ rad/s by the catch angle, while a nudge moves it much faster. Catching above 0.6 rad/s therefore reacts instantly to pushes without interrupting natural falls. Without this rule a 1.0 N push knocked the pole over; with it, 1.2 N is survived.
- **Letting go is judged against the requested lean,** not vertical, so the robot can let go even while braking or returning home.

The cost is real: letting go reduces the margin for a push. For the best disturbance rejection you would not use this cycle, and it can be reduced by lowering `catch_tilt_rad`.

Part 7: Test results and known limits

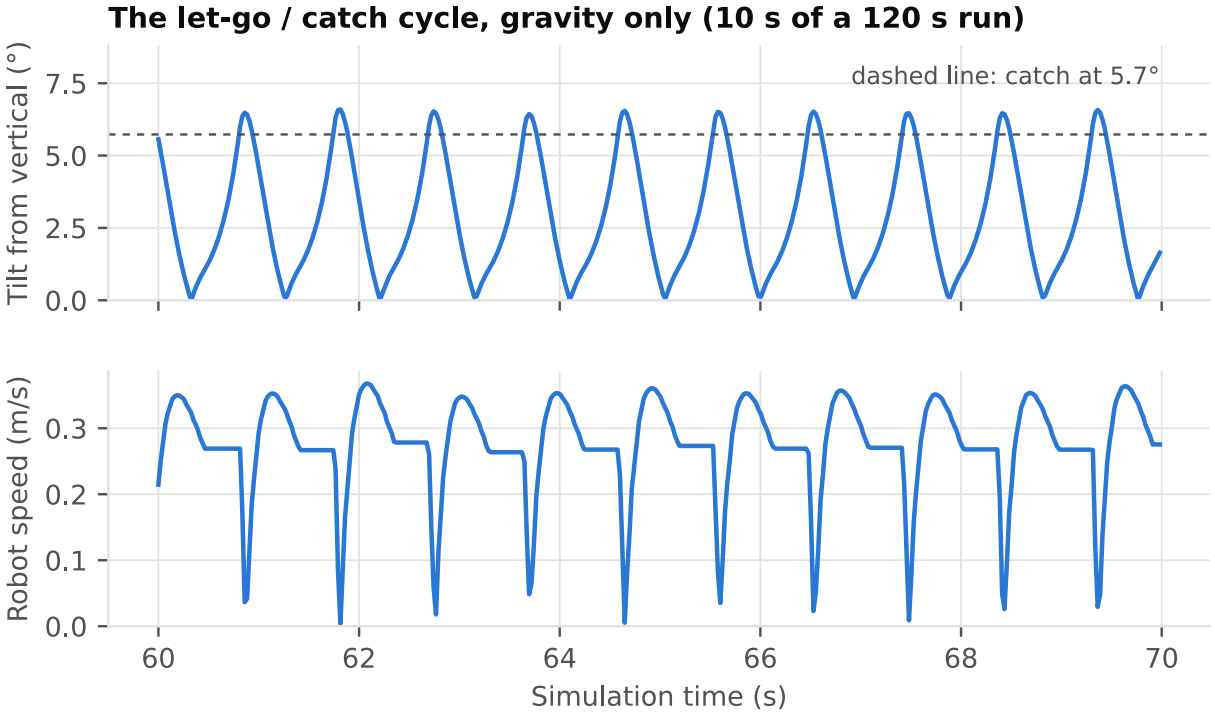
All results come from headless simulation runs, each isolated with its own `GZ_PARTITION` and `ROS_DOMAIN_ID`. Figures use data recorded from the current code. **No test has been run on real hardware.**

7.1 Gravity is modelled correctly

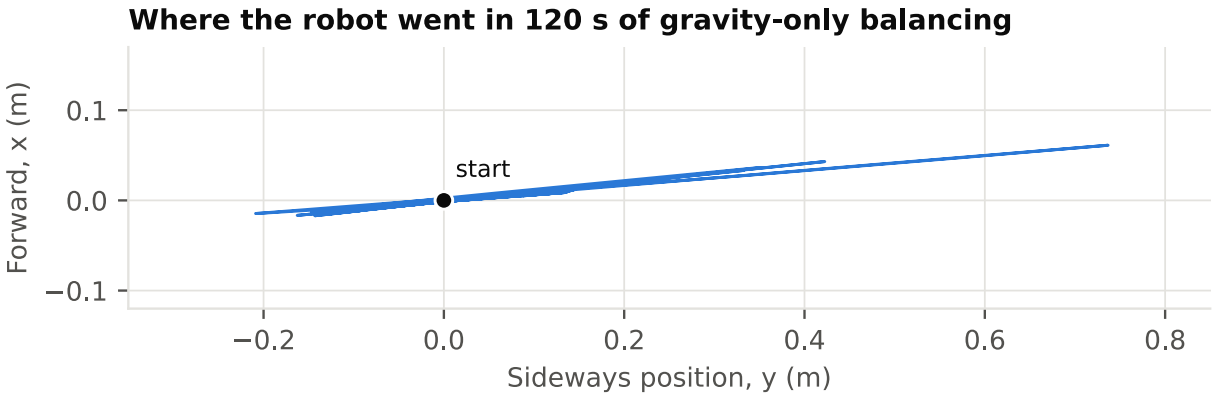


With the controller off, the pole went from 1° to 90° in 0.877 s, against 0.896 s from theory for a fixed pivot. The curves match closely until large angles, where the simulated pole is slightly faster because the robot's base can recoil.

7.2 The cycle with gravity only

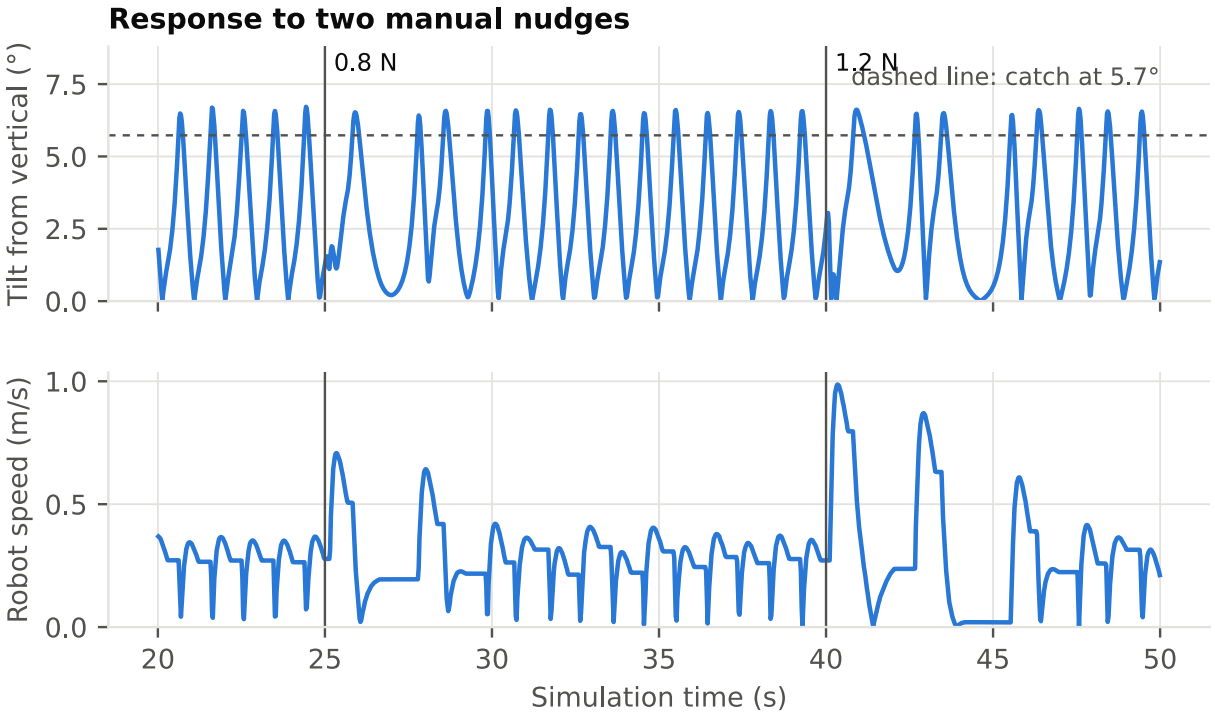


Tilt rises under gravity, slightly passes the catch angle, and is driven back, about once every 0.95 s. Over 120 s there was no fall, and the pole never settled. The robot averaged 0.27 m/s and peaked at 0.72 m/s.



Floor space. In this run the robot stayed within 0.74 m of its start and moved almost entirely along one line, since the cycle keeps repeating along the direction of the first fall. That direction is random, so plan the mocap area for this distance in any direction. This is one run of 120 s, and longer runs may differ.

7.3 Response to manual pushes



Each push makes the robot react at once and travel further for a few seconds, and then the regular cycle resumes.

Push at the top of the pole (0.5 m)	Result, 8 directions tested	Largest tilt
0.8 N	Recovered in all 8	6.6°
1.0 N	Recovered in all 8	11.0°
1.2 N	Recovered in all 8	14.6°
1.5 N	**Fell**, on the second direction tested (45°)	–

Two further 2-minute runs took 36 random pushes of 0.3–1.0 N, from the automatic nudges used at the time, with no falls. The same controller was used; only the automatic nudges have since been removed.

7.4 What set the acceleration limit

Velocity lag while accelerating to 1 m/s, measured with ground-truth odometry (this includes message delay):

Direction	2 m/s^2	5 m/s^2	8 m/s^2
Forward	0.06 m/s	0.12 m/s	0.31 m/s
Sideways	0.03 m/s	0.08 m/s	0.25 m/s
Diagonal	0.03 m/s	0.09 m/s	0.42 m/s

Limits of 4, 5, 6 and 8 m/s^2 were compared by the strongest push survived, and 6 m/s^2 was best. Those tests were made before the let-go cycle existed.

7.5 What the lean limit fixed

Lean in the opposite direction after a strong push, measured before the let-go cycle existed:

Push	Without lean limit	With lean limit
1.2 N	7.0–7.5°	4.8°
1.5 N	9.6–12.0°	4.8°
1.75 N (straight)	13.0°	4.8°

The cost was longer travel after a push (up to about 2.2 m). A 2° limit couldn't brake enough, and the robot ran away.

7.6 Known limits

- **Simulated sensors are perfect.** Angles and rates come directly from the physics engine, with no noise or filtering, and delay is limited to message passing. Real results will be worse.
- **Mecanum wheels are approximated** with friction, not rollers. Real traction, roller vibration and motor limits will differ.
- **The controller has no velocity feedback,** as explained in 6.5.
- **Limits and gains are tuned to this simulation,** including a 2 m/s limit that may exceed what a real LIMO can do.
- **Linear model.** The gains assume small angles. The 34° cut-off is a give-up point, not a tested recovery limit.

- **The AgileX repository declares no license,** so the LIMO meshes aren't covered by this project's Apache-2.0 license. Check with AgileX.
- **Placeholders remain.** The maintainer details in `setup.py` and `package.xml` have not been filled in.

Part 8: How the project developed

Each stage started from something observed in the simulation, was diagnosed with measurements, and then fixed.

#	What was observed or asked for	What was found	Decision
1	First draft of the robot and controller	Tilt-to-velocity control can't balance (6.2); wheels couldn't strafe; time source was wrong	Acceleration-based LQR, anisotropic friction, simulation time
2	Use the AgileX LIMO	No mecanum model, and only Gazebo Classic plugins	Keep its shapes and dimensions; rebuild the drive for Harmonic; custom world
3	Robot missing from Gazebo's tree	A leftover server; the window connected to the wrong one	Run server and window as one process
4	Pole floating in mid-air	LIMO's top isn't flat; pivot was 11 cm above the body	Measure the mesh; later, build a platform
5	“Force Quit / Wait” dialog	Meshes with 1,619 parts and over 750,000 triangles	Merge by colour and simplify: 63.6 MB to 3.8 MB
6	Pole looks stiff	Joints were free; it just never leaned more than 2–3°	Free-fall start, softer gains
7	Disturbance should be a human nudge	Diagonal pushes failed first; stiffer gains made it worse. Ground truth showed the wheels couldn't follow 8 m/s^2	Finger-tap model; acceleration limit matched to the wheels; 2D limits; ground-truth odometry
8	Still looks stiff; arrow keys wanted	Gravity matched theory. Camera at 6 m made motion look tiny	Larger free fall, closer camera, key input
9	Robot overshoots after strong pushes	Braking leaned the pole further than the push had; the limit on it was missing	Cascade with a lean limit and return-speed limit
10	Model broken after a commit	Pasted text and an old model version had been committed	Restored from git history
11	Pole should keep trying to fall	Controller held it under 0.1°. Letting go lost nudge margin, and the first catch at 10° was marginal	Let-go / catch cycle; fast-motion catch; first catch at 8°; wrench clearing
12	Prepare for a real robot	Random nudges have no real-world equivalent	Removed; one start-up nudge plus manual pushes remain

8.1 Design decisions and alternatives

Decision	Alternative	Why this choice
Velocity commands through MecanumDrive	Controlling wheel torque	Simpler, and it matches the velocity-command interface used by the bridge and `/cmd_vel`
Friction-based mecanum wheels	Modelling rollers, or moving the body directly	Rollers are slow and fragile. Moving the body directly would hide traction, the very limit found in stage 7
LQR in cascade form	Separate PID loops	Gains come from the physics, and the cascade adds meaningful limits
Commanded velocity as the estimate	Wheel odometry	Smooth with no delay; wheel odometry can't see slip either. Its risk is covered in 6.5
Torque at the pivot for nudges	Force at the contact point	The ROS wrench message has no offset for the force. F × h tips the pole in the same way
Single-process Gazebo	Separate server and window	A separate window can attach to the wrong server
Gazebo and terminal key input	Only one	The terminal is a fallback. One shared topic keeps one code path

8.2 Next step: the real robot in the mocap room

This is a plan and hasn't been tested. `balance_controller.py` is written so most of it carries over; the interfaces and tuning do not.

Area	Simulation	Real robot
Pole angles	Joint states from Gazebo	A new node computes pole tilt relative to the robot from mocap rigid bodies, and publishes pitch and roll as `/joint_states` with the same joint names
Tilt rates	Exact, from physics	Differentiated from mocap and filtered, which adds noise and delay
Clock	`use_sim_time: true`	Off; check the timestamps the controller's time step relies on
Nudges	`disturbance.py`	Not used: a person touches the pole. Start with the pole held upright
Limits	2 m/s, 6 m/s^2	Measure the real robot and set `max_vel` and `max_accel` to match
Gains	From the simulated pole	Recalculate from the real pole's mass, length and inertia (6.1, 6.4)
Let-go cycle	Catch at 5.7°	Start small or close to continuous control. Real noise will likely keep the pole moving anyway
Safety	None needed	A clear area, a low speed limit at first, a way to stop the robot immediately, and a pole that can fall safely. Confirm the stop behaviour before balancing

Part 9 — Line-by-line code walkthrough

Each source line is shown with its line number. The grey box under a line (or under a group of lines) explains what it does and why. Blank lines are shown but not explained; docstrings and closing brackets are explained as a group.

balance_controller.py

`src/omni_pendulum_sim/omni_pendulum_sim/balance_controller.py` · 237 lines

The brain of the robot. It subscribes to the pendulum angles on /joint_states and publishes a velocity command on /cmd_vel. Read it in this order: `AxisController` holds the control maths for one tilt direction, `limit_norm` applies limits, and `joint_callback` is the control loop that runs the free-fall start, the safety cut-off and the let-go / catch cycle.

```
"""
Balance a 2-DOF inverted pendulum on a mecanum (omnidirectional) AgileX LIMO.

Each tilt axis is treated as an independent cart-pole:
  pitch (tilt toward +X)  -> drive the base along X
  roll  (tilt toward -Y)  -> strafe the base along Y
so whichever way the pole starts to fall, the robot drives in that direction to
get back underneath it.

The base only accepts velocity commands (/cmd_vel), but a pendulum is balanced
by *accelerating* under it, so the controller computes a desired acceleration
and integrates it into a velocity command. It is an LQR state feedback written
as a cascade, so the parts can be limited separately:

  lean_ref = -(k_pos * (p - p_ref) + k_vel * (v - v_ref)) / k_theta
  a = k_theta * (theta - lean_ref) + k_omega * theta_dot
  v += a * dt,  p += v * dt

The inner line keeps the pole up and is never limited. The outer line is how
the robot slows down and returns home: to brake it has to tip the pole slightly
backwards first (an inverted pendulum cannot be stopped any other way). Capping
lean_ref at max_lean_rad stops the robot from braking so hard after a big
nudge that the pole swings over to the other side and falls; capping the
position term at max_return_speed keeps the drive back home gentle.

At startup the controller stays idle and lets the pole fall freely until it
has tipped past engage_tilt_rad, then catches it.

An upright pole held by a good controller barely moves, so balancing runs as a
repeating cycle: once the pole is back under control the robot lets go (no
acceleration) and gravity tips the pole again; at catch_tilt_rad the robot
drives under it and brings it back.

The default gains come from a deliberately soft LQR design (so the pole visibly
sways while the robot works to keep it up) on the linearised model
  theta_ddot = (m g l / J) * theta - (m l / J) * a
for the pendulum in robot.urdf.xacro (rod 0.5 m / 0.1 kg + 0.1 kg ball on top).
"""
```

Lines 1–38 The module docstring: a plain-English summary of the whole controller, kept at the top of the file so anyone opening it understands the idea before reading code. It states the three key ideas: (1) each tilt axis is treated as its own cart-pole, (2) the base takes velocity commands, so the controller computes an acceleration and integrates it into a velocity, (3) the control law is an LQR state feedback rewritten as a two-level cascade so that braking can be limited without ever limiting the part that keeps the pole up, and (4) balancing runs as a repeating let-go / catch cycle so that gravity keeps visibly tipping the pole. It also records where the default gains came from (the linearised pendulum model).

```

import math
```

Standard-library maths module. Used for `math.hypot` (length of a 2D vector) and `math.degrees` (radians to degrees for log messages).

```
41
42 from geometry_msgs.msg import Twist
```

`Twist` holds a linear and an angular velocity. It is the command format of Gazebo's MecanumDrive system: `linear.x` = forward speed, `linear.y` = sideways speed, both in m/s in the robot frame.

```
43 import rclpy
```

`rclpy` is the ROS 2 client library for Python. It provides `init`, `spin` and `shutdown`, which start the ROS middleware, process incoming messages and stop cleanly.

```
44 from rclpy.node import Node
```

`Node` is the base class of every ROS 2 program. Subclassing it gives this controller a name on the ROS graph, parameters, publishers, subscribers, timers and a logger.

```
45 from sensor_msgs.msg import JointState
```

`JointState` is the standard message carrying joint names, positions, velocities and efforts. Gazebo's JointStatePublisher system sends one for every joint of the robot, including the two pendulum joints; ros_gz_bridge converts it to this ROS type.

```
46
47
48 class AxisController:
```

One `AxisController` object handles one tilt direction. The same class is used twice (X and Y) because the pendulum is symmetric and a mecanum base can move along X and Y independently, so the two-dimensional problem splits into two identical one-dimensional cart-pole problems.

```
49     """LQR balancer for one tilt axis."""
```

Docstring for the class.

```
50
51     def __init__(self, k_theta, k_omega, k_pos, k_vel):
```

Constructor. It receives the four LQR gains that define how strongly the controller reacts to each part of the state.

```
52         self.k_theta = k_theta
```

`k_theta`: how many m/s^2 of base acceleration to command per radian of pole tilt. This is the main 'get underneath the pole' term.

```
53         self.k_omega = k_omega
```

`k_omega`: acceleration per rad/s of tilt rate. It reacts to how fast the pole is falling, which damps the motion and stops the base from oscillating around the pole.

```
54         self.k_pos = k_pos
```

`k_pos`: how strongly the robot is pulled back to its reference position (its starting point unless you drive it with /cmd_vel_ref).

```
55         self.k_vel = k_vel
```

`k_vel`: how strongly the robot's velocity is pulled toward the reference velocity (zero when standing). This is what makes it slow down after catching the pole.

```
56         self.reset()
```

Start from a clean state (zero velocity and position).

```
57
58     def reset(self):
```

`reset` clears the controller's internal memory. It is called at start-up and every time the pendulum has fallen, so the next attempt does not inherit a stale velocity.

```
59         self.vel = 0.0
```

`self.vel` is the velocity the controller has commanded so far. The base is velocity-controlled and was measured (with ground-truth odometry) to follow commands closely within the acceleration limit, so the commanded velocity doubles as the controller's estimate of the real base velocity.

```
60          self.pos = 0.0
```

`self.pos` is the integral of the commanded velocity: how far the controller believes the robot has travelled from where balancing started.

```
61          self.pos_ref = 0.0
```

`self.pos_ref` is where the robot should be. It stays at 0 unless someone publishes a desired velocity on /cmd_vel_ref, in which case it moves along at that velocity.

```
62
63      def lean_ref(self, vel_ref, max_return_speed, dt):
```

The outer loop. It answers: 'how much should the pole lean so that the robot slows down / goes home?' To decelerate, an inverted pendulum on a cart must first be tipped slightly backwards, so braking is expressed as a desired lean angle.

```
64          """Lean the pole should have to brake / return home (before limiting)."""
```

Docstring: the returned lean is the raw request, before the caller applies the `max_lean_rad` cap.

```
65          self.pos_ref += vel_ref * dt
```

Advance the reference position by the reference velocity times the time step (simple Euler integration). With no /cmd_vel_ref this adds zero.

```
66          pos_term = self.k_pos * (self.pos - self.pos_ref)
```

Position error term: gain times how far the robot is from where it should be.

```
67          pos_limit = self.k_vel * max_return_speed
```

Largest allowed position term. In steady state the outer loop balances `k_pos*error` against `k_vel*velocity`, so capping the position term at `k_vel*max_return_speed` means the robot can never drive home faster than `max_return_speed` (0.3 m/s), however far a nudge carried it.

```
68          pos_term = max(min(pos_term, pos_limit), -pos_limit)
```

Clamp the position term to plus/minus that limit.

```
69          return -(pos_term + self.k_vel * (self.vel - vel_ref)) / self.k_theta
```

The LQR control law rearranged: a lean of `-(k_pos*e_p + k_vel*e_v)/k_theta` produces exactly the same acceleration as the original four-term LQR law, but now the 'braking' part is a single number (an angle) that can be limited. The minus sign: if the robot is moving forward (positive velocity error), the pole must lean backwards (negative angle) so the robot decelerates.

```
70
71      def accel(self, theta, theta_dot, lean_ref):
```

The inner loop: the acceleration that keeps the pole up.

```
72          return self.k_theta * (theta - lean_ref) + self.k_omega * theta_dot
```

`a = k_theta*(theta - lean_ref) + k_omega*theta_dot`. If the pole tilts forward more than requested, accelerate forward under it; the `theta_dot` term reacts to the falling speed and adds damping. This part is never limited except by the physical acceleration limit, because limiting it would let the pole fall.

```
73
74      def integrate(self, accel, dt):
```

Turns the chosen acceleration into a new velocity and position.

```
75          self.vel += accel * dt
```

Euler integration: new velocity = old velocity + acceleration * time step. This is why the controller can command acceleration even though the base only accepts velocity.

```
76          self.pos += self.vel * dt
```

Integrate velocity into position, giving the position estimate used by the outer loop.

```
77
78
79 def limit_norm(x, y, limit):
```

Helper that limits the length of a 2D vector while keeping its direction. Used for the lean, acceleration and speed limits so that diagonal motions are scaled as a whole instead of being clipped per axis (per-axis clipping would bend a diagonal command toward an axis).

```
80     """Scale the vector (x, y) down to length `limit`, keeping its direction."""
```

Docstring.

```
81     norm = math.hypot(x, y)
```

Length of the vector, `sqrt(x*x + y*y)`.

```
82     if norm <= limit:
```

If it is already within the limit...

```
83         return x, y
```

...return it unchanged.

```
84     return x * limit / norm, y * limit / norm
```

Otherwise scale both components by `limit/norm`, which shortens the vector to exactly `limit` without changing its direction.

```
85
86
87 class OmnidirectionalBalanceController(Node):
```

The ROS 2 node class. It owns the two axis controllers, reads parameters, subscribes to sensor topics and publishes velocity commands.

```
88
89     def __init__(self):
```

Constructor, runs once when the node starts.

```
90         super().__init__('balance_controller')
```

Registers the node on the ROS graph under the name `balance_controller` (visible with `ros2 node list`).

```
91
92         self.declare_parameter('k_theta', 33.4)
```

Declares the ROS parameter `k_theta` with default 33.4. The four gains come from an LQR design with Q = diag(20, 1, 0.5, 1) and R = 0.3 on the linearised pendulum model. Declaring them as parameters means they can be changed at launch (`--ros-args -p k_theta:=...`) without editing code.

```
93         self.declare_parameter('k_omega', 7.1)
```

Tilt-rate gain `k_omega` = 7.1.

```
94         self.declare_parameter('k_pos', 1.3)
```

Position gain `k_pos` = 1.3.

```
95         self.declare_parameter('k_vel', 3.1)
```

Velocity gain `k_vel` = 3.1.

```
96         # Hard safety limit; normally the lean limit keeps the robot well below it
```

Comment explaining the next parameter.

```
97         self.declare_parameter('max_vel', 2.0)
```

`max_vel` = 2.0 m/s: an absolute speed limit. The robot never goes faster, even if the controller asks.

```
98         # How far the controller may deliberately tip the pole to brake or
99         # return home, and how fast it drives back to where it started
```

Lines 98–99 Comment explaining the next two parameters.

```
100         self.declare_parameter('max_lean_rad', 0.06)
```

`max_lean_rad` = 0.06 rad (3.4 degrees): the most the controller may deliberately tip the pole when braking or returning home. This is the fix for the 'robot runs too fast and the pole leans to the other side' problem: in tests before the let-go cycle was added, the opposite lean stayed under 5 degrees for every survivable nudge. Below 0.05 rad the robot cannot brake hard enough and runs away.

```
101         self.declare_parameter('max_return_speed', 0.3)
```

`max_return_speed` = 0.3 m/s: how fast the robot drives back to its starting point after a nudge.

```
102         # What the mecanum wheels can actually deliver in any direction (the
103         # diagonal is the weakest). The controller assumes the base follows its
104         # commands, so asking for more than this makes it lose track and overshoot.
```

Lines 102–104 Comment explaining why the acceleration limit exists.

```
105         self.declare_parameter('max_accel', 6.0)
```

`max_accel` = 6.0 m/s^2: the most acceleration the controller will ask for. Tests with ground-truth odometry showed the mecanum wheels cannot accelerate the robot much harder (especially diagonally). When the controller asked for 8 m/s^2, the real robot lagged behind, the controller's velocity estimate became wrong, it braked too late and dropped the pole. 4, 5, 6 and 8 were compared and 6 gave the best nudge tolerance.

```
106         self.declare_parameter('max_tilt_rad', 0.6)
```

`max_tilt_rad` = 0.6 rad (34 degrees): beyond this the pole cannot be saved, so the controller gives up and stops the base instead of chasing a fallen pole across the floor.

```
107         self.declare_parameter('engage_tilt_rad', 0.14)
```

`engage_tilt_rad` = 0.14 rad (8 degrees): at start-up the controller stays idle until the pole has fallen this far, so you can see gravity act first. 10 degrees was tried and proved marginal: the first catch occasionally failed at the 2 m/s speed limit.

```
108         self.declare_parameter('control_period', 0.005)
```

`control_period` = 0.005 s: run the control law at most every 5 ms (200 Hz). Joint states arrive about every millisecond; 200 Hz is far faster than the pendulum dynamics (time constant about 0.2 s) and keeps CPU load low.

```
109         # Let-go / catch cycle: catch at this tilt, or sooner if the pole moves faster
110         # than gravity alone would make it (a nudge). Let go again once the pole is
111         # within release_tilt_rad of the requested lean and slower than release_rate
```

Lines 109–111 Comment explaining the let-go / catch cycle parameters below.

```
112         self.declare_parameter('catch_tilt_rad', 0.1)
```

`catch_tilt_rad` = 0.1 rad (5.7 degrees): during the cycle, the robot lets gravity tip the pole this far before catching it. Larger values give bigger visible swings but leave less margin for a nudge.

```
113         self.declare_parameter('catch_rate', 0.6)
```

`catch_rate` = 0.6 rad/s: catch immediately if the pole is moving faster than this. Falling under gravity alone, the pole only reaches about 0.47 rad/s by the catch angle, while a nudge moves it much faster. So this reacts at once to nudges without cutting natural falls short. It brought back the nudge margin that letting go had cost.

```
114         self.declare_parameter('release_tilt_rad', 0.015)
```

`release_tilt_rad` = 0.015 rad (0.9 degrees): the pole must be this close to the requested lean before the robot lets go.

```
115         self.declare_parameter('release_rate', 0.1)
```

`release_rate` = 0.1 rad/s: and it must also be moving slower than this.

```
116
117         gains = [self.get_parameter(n).value for n in ('k_theta', 'k_omega', 'k_pos', 'k_vel')]
```

Read the four gain parameters into a list (their values may have been overridden at launch).

```
118         self.x_axis = AxisController(*gains)
```

Create the X-axis controller (pitch tilt, forward/back motion).

```
119         self.y_axis = AxisController(*gains)
```

Create the Y-axis controller (roll tilt, sideways motion) with the same gains.

```
120         self.max_vel = self.get_parameter('max_vel').value
121         self.max_accel = self.get_parameter('max_accel').value
122         self.max_lean = self.get_parameter('max_lean_rad').value
123         self.max_return_speed = self.get_parameter('max_return_speed').value
124         self.max_tilt_rad = self.get_parameter('max_tilt_rad').value
125         self.engage_tilt_rad = self.get_parameter('engage_tilt_rad').value
126         self.control_period = self.get_parameter('control_period').value
```

Lines 120–126 Read the remaining parameters into attributes so they are fast to access inside the control loop.

```
127         self.catch_tilt_rad = self.get_parameter('catch_tilt_rad').value
128         self.catch_rate = self.get_parameter('catch_rate').value
129         self.release_tilt_rad = self.get_parameter('release_tilt_rad').value
130         self.release_rate = self.get_parameter('release_rate').value
```

Lines 127–130 Read the cycle parameters into attributes.

```
131
132         # Optional user velocity request (e.g. from teleop_twist_keyboard)
```

Comment: the next two attributes allow an external program to request a travel velocity while balancing.

```
133         self.vel_ref_x = 0.0
```

Requested forward velocity from /cmd_vel_ref (0 by default).

```
134         self.vel_ref_y = 0.0
```

Requested sideways velocity from /cmd_vel_ref.

```
135         self.last_stamp = None
```

Simulation time of the last control step; `None` until the first message arrives.

```
136         self.engaged = False
```

`engaged` becomes True once the pole has passed the engage angle and balancing has started.

```
137         self.catching = True
```

`catching` is True while the controller is actively driving under the pole and False while it has let go. It starts True so the first catch acts as soon as the pole passes the engage angle.

```
138         self.fallen = False
```

`fallen` remembers whether the pole is currently past the safety limit, so the warning is logged only once per fall.

```
139
140         self.create_subscription(JointState, '/joint_states', self.joint_callback, 10)
```

Subscribe to /joint_states. Every incoming message calls `joint_callback`. The `10` is the queue depth: up to 10 messages are buffered if the callback is busy.

```
141         self.create_subscription(Twist, '/cmd_vel_ref', self.ref_callback, 10)
```

Subscribe to /cmd_vel_ref, an optional desired travel velocity (for example from teleop_twist_keyboard remapped to this topic).

```
142         self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
```

Create the publisher for /cmd_vel. ros_gz_bridge forwards it to Gazebo's MecanumDrive system, which turns it into wheel speeds.

```
143
144         self.get_logger().info('Balance controller started')
```
Log a start-up message to the terminal.
```
145
146     def ref_callback(self, msg):
```
Callback for /cmd_vel_ref messages.
```
147         self.vel_ref_x = msg.linear.x
```
Store the requested forward velocity.
```
148         self.vel_ref_y = msg.linear.y
```
Store the requested sideways velocity.
```
149
150     def joint_callback(self, msg):
```
The main control loop. It runs every time Gazebo publishes joint states (about 1000 times per second of simulated time) and decides the next velocity command.
```
151         try:
```
The joint order in the message is not guaranteed, so look the pendulum joints up by name.
```
152             pitch_idx = msg.name.index('pendulum_pitch_joint')
```
Index of the pitch joint (tilt about the robot's Y axis, i.e. leaning forward/back).
```
153             roll_idx = msg.name.index('pendulum_roll_joint')
```
Index of the roll joint (tilt about the robot's X axis, i.e. leaning left/right).
```
154         except ValueError:
```
If a joint is missing (for example a message published before the robot was fully spawned)...
```
155             return
```
...ignore this message.
```
156
157         theta_pitch = msg.position[pitch_idx]
```
Pitch angle in radians. Positive means the top of the pole leans toward +X (forward).
```
158         theta_roll = msg.position[roll_idx]
```
Roll angle in radians. Positive means the top of the pole leans toward -Y (right).
```
159         omega_pitch = msg.velocity[pitch_idx] if msg.velocity else 0.0
```
Pitch angular velocity in rad/s, measured directly by Gazebo (no noisy numerical differentiation needed). Falls back to 0 if the message carries no velocities.
```
160         omega_roll = msg.velocity[roll_idx] if msg.velocity else 0.0
```
Roll angular velocity in rad/s.
```
161
162         # Use the message's simulation timestamp so dt matches simulated physics
```
Comment: why the message timestamp is used.
```
163         stamp = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9
```
Convert the message's header stamp (seconds + nanoseconds of simulation time) to seconds. Using simulation time means dt is correct even when Gazebo runs slower or faster than real time.
```
164         if self.last_stamp is None:
```
On the very first message there is no previous time...
```
165             self.last_stamp = stamp
```
...so remember this time...
```
166             return
```
...and wait for the next message.

```
        dt = stamp - self.last_stamp
```

Time elapsed since the last control step.

```
        if dt < self.control_period:
```

If less than `control_period` has passed...

```
            return
```

...skip this message (rate limiting to 200 Hz).

```
        self.last_stamp = stamp
```

Remember the time of this control step.

```

        cmd = Twist()
```

Create an all-zero velocity command. It is filled in later, or published as-is to stop the robot.

```

        # Let the pole fall freely until it has tipped far enough, then catch it
```

Comment: free-fall phase at start-up.

```
        if not self.engaged:
```

While balancing has not started yet...

```
            if max(abs(theta_pitch), abs(theta_roll)) < self.engage_tilt_rad:
```

...check whether the pole is still within the engage angle on both axes...

```
                self.pub.publish(cmd)
```

...if so, keep the robot still (publish zero velocity)...

```
                return
```

...and do nothing else, letting gravity tip the pole.

```
            self.engaged = True
```

The pole has fallen past the engage angle: start balancing from now on.

```
            self.get_logger().info(
                f'Pole tipped past {math.degrees(self.engage_tilt_rad):.0f} deg, balancing')
```

Lines 180–181 Log the moment balancing starts, e.g. 'Pole tipped past 8 deg, balancing'.

```

        # Safety cutoff once the pendulum has fallen past recovery
```

Comment: safety cut-off.

```
        if abs(theta_pitch) > self.max_tilt_rad or abs(theta_roll) > self.max_tilt_rad:
```

If either angle exceeds the recoverable limit (34 degrees)...

```
            if not self.fallen:
```

...and this is the first control step since it fell...

```
                self.get_logger().warn('Pendulum fell over, stopping base')
```

...log a warning once...

```
                self.fallen = True
```

...and remember that it is down.

```
            self.x_axis.reset()
```

Clear the X controller's memory.

```
            self.y_axis.reset()
```

Clear the Y controller's memory.

```
            self.pub.publish(cmd)
```

Stop the robot.

```
            return
```

Skip the control law.

```
        self.fallen = False
```

The pole is within limits, so clear the fallen flag.

```

        # Positive pitch tips toward +X; positive roll tips toward -Y, so the
        # y axis works in negated coordinates. Limits are applied to the 2D
        # vectors so diagonal moves keep their direction.
```

Lines 194–196 Comment explaining the sign convention and the 2D limits used below.

```
        lean_x = self.x_axis.lean_ref(self.vel_ref_x, self.max_return_speed, dt)
```

Outer loop for X: the lean needed to brake/return along X.

```
        lean_y = self.y_axis.lean_ref(-self.vel_ref_y, self.max_return_speed, dt)
```

Outer loop for Y. The Y axis works in negated coordinates (because positive roll tips toward -Y), so the requested Y velocity is negated on the way in.

```
        lean_x, lean_y = limit_norm(lean_x, lean_y, self.max_lean)
```

Cap the combined requested lean at `max_lean_rad`. This is the key anti-overshoot limit.

```

        # Gravity takes over while letting go; the robot just keeps its velocity
```

Comment: while letting go, the commanded acceleration is zero, so only gravity moves the pole.

```
        rate = math.hypot(omega_pitch, omega_roll)
```

How fast the pole is moving, combining both axes.

```
        if self.catching:
```

If currently catching, decide whether to let go...

```
            error = math.hypot(theta_pitch - lean_x, theta_roll - lean_y)
```

...by measuring how far the pole is from the lean the outer loop asked for (not from vertical), so the robot can let go while still braking or returning home...

```
            self.catching = error > self.release_tilt_rad or rate > self.release_rate
```

...and keep catching until the pole is both close to that lean and moving slowly.

```
        else:
```

If currently letting go...

```
            tilt = math.hypot(theta_pitch, theta_roll)
```

...measure the tilt from vertical...

```
            self.catching = tilt > self.catch_tilt_rad or rate > self.catch_rate
```

...and start catching once the tilt passes `catch_tilt_rad`, or sooner if a nudge has made the pole move faster than `catch_rate`.

```

        ax = ay = 0.0
```

Default: no acceleration. While letting go, this zero means the robot simply keeps its current velocity.

```
        if self.catching:
```

Only compute an acceleration while catching.

```
            ax = self.x_axis.accel(theta_pitch, omega_pitch, lean_x)
```

Inner loop for X: acceleration from pitch angle, pitch rate and the requested lean.

```
            ay = -self.y_axis.accel(theta_roll, omega_roll, lean_y)
```

Inner loop for Y, negated back into robot coordinates (+Y = left).

```
            ax, ay = limit_norm(ax, ay, self.max_accel)
```

Cap the combined acceleration at what the wheels can deliver (`max_accel`), keeping the direction.

```
        self.x_axis.integrate(ax, dt)
```

Integrate the X acceleration into the X velocity and position.

```
        self.y_axis.integrate(-ay, dt)
```

Integrate the Y acceleration (negated again into the controller's internal Y convention).

```

        vx, vy = limit_norm(self.x_axis.vel, -self.y_axis.vel, self.max_vel)
```

Cap the combined speed at `max_vel`, keeping the direction.

```
        self.x_axis.vel, self.y_axis.vel = vx, -vy
```

Write the limited velocities back into the controllers. Without this, the internal velocity would keep growing beyond what is actually commanded (integrator wind-up) and the controller would lose track of the real robot.

```
        cmd.linear.x = vx
```

Forward velocity command.

```
        cmd.linear.y = vy
```

Sideways velocity command.

```
        self.pub.publish(cmd)
```

Send the command to /cmd_vel.

```


def main(args=None):
```

`main` is the entry point that `ros2 run omni_pendulum_sim balance_controller` calls (registered in setup.py).

```
    rclpy.init(args=args)
```

Initialise ROS 2 for this process.

```
    node = OmnidirectionalBalanceController()
```

Create the controller node (runs `__init__`).

```
    try:
```

Start a block that handles Ctrl+C cleanly.

```
        rclpy.spin(node)
```

`spin` hands control to ROS: it waits for messages and calls the callbacks until shutdown.

```
    except KeyboardInterrupt:
```

Ctrl+C raises KeyboardInterrupt...

```
        pass
```

...which is expected, so it is ignored instead of printing a traceback.

```
    node.destroy_node()
```

Release the node's publishers and subscriptions.

```
    rclpy.try_shutdown()
```

Shut ROS down. `try_shutdown` does nothing if the launch system has already shut it down, avoiding an error on Ctrl+C.

```


if __name__ == '__main__':
```

Standard Python guard: only call `main` when this file is run directly...

```
    main()
```

...not when it is imported by another module.

disturbance.py

`src/omni_pendulum_sim/omni_pendulum_sim/disturbance.py` · 169 lines

Simulation only: it stands in for a person touching the pole. It gives one tiny start-up nudge, then turns arrow keys and /nudge messages into short pushes. `tick` runs every 10 ms and shapes the push over time. On a real robot this node is not used.

```
"""
Nudge the pendulum like a person tapping it with a finger.

A nudge is a short horizontal push on the pole: the force rises and falls as a
half-sine over nudge_duration seconds and acts at a point somewhere along the
upper part of the rod. Gazebo's ApplyLinkWrench system (bridged by
ros_gz_bridge) only takes a wrench at the link origin, which is the pivot, so
the push is applied as the equivalent torque: force * contact height.

Nudges happen:
  * once at startup (a very small one) so the pole starts to fall on its own;
    after that gravity keeps the let-go / catch cycle going without any nudges
  * from the arrow keys, either in the Gazebo window (Key Publisher plugin) or
    in a terminal running keyboard_nudge; both publish Qt key codes on
    /keyboard/keypress. Up/Down push along +X/-X (robot forward/back),
    Left/Right along +Y/-Y, and +/- change the push strength
  * on demand, by publishing the push force (N) to /nudge, e.g.
      ros2 topic pub --once /nudge geometry_msgs/msg/Vector3 "{x: 0.5, y: 0.0}"

Gazebo adds persistent wrenches together rather than replacing them, so this
node only ever publishes the *change* in total torque since the last update,
and clears the accumulated wrenches shortly after each nudge ends.
"""
```

Lines 1–23 Module docstring. It explains the physical model of a nudge (a half-sine force pulse at some height on the rod), why it is applied as a torque (Gazebo's ApplyLinkWrench can only apply a wrench at the link origin, which is the pivot, so a force F at height h becomes the equivalent torque F*h), the ways a nudge can be triggered (one automatic start-up nudge, then only manual ones), and the implementation detail that Gazebo adds persistent wrenches together, so the node sends only the change in torque and clears the stack after each nudge.

```

import math
```

Maths module: `sin`, `cos`, `atan2`, `hypot`, `pi`, `inf`, `degrees`.

```
import random
```

Random numbers for random nudge timing, strength, direction and contact height.

```

from geometry_msgs.msg import Vector3
```

`Vector3` (x, y, z) message, used on /nudge for a manual push force in newtons.

```
import rclpy
```

ROS 2 Python client library.

```
from rclpy.node import Node
```

Base class for ROS 2 nodes.

```
from ros_gz_interfaces.msg import Entity, EntityWrench
```

`Entity` identifies something in the Gazebo world (here: the pendulum link); `EntityWrench` is a force + torque to apply to an entity. Both come from ros_gz_interfaces and are bridged to the matching Gazebo messages.

```
from std_msgs.msg import Int32
```

`Int32` message, used for key codes on /keyboard/keypress.

```

```

```
# Qt key codes, as published by Gazebo's Key Publisher GUI plugin
```

Comment: the next constants are Qt key codes.

```
KEY_LEFT, KEY_UP, KEY_RIGHT, KEY_DOWN = 0x01000012, 0x01000013, 0x01000014, 0x01000015
```

Qt key codes for the arrow keys. Gazebo's GUI is built with Qt, and its Key Publisher plugin publishes these numbers when a key is pressed. keyboard_nudge.py publishes the same numbers, so both input methods share one code path.

```
KEY_DIRECTIONS = {KEY_UP: (1.0, 0.0), KEY_DOWN: (-1.0, 0.0),
                  KEY_LEFT: (0.0, 1.0), KEY_RIGHT: (0.0, -1.0)}
```

Lines 36–37 Maps each arrow key to a unit push direction in the robot frame: Up = +X (forward), Down = -X, Left = +Y, Right = -Y. The default Gazebo camera looks along +X, so screen-up matches robot-forward.

```
KEY_STRONGER = (ord('+'), ord('='))
```

Key codes that increase the push strength: '+' and '=' (the same key without Shift).

```
KEY_WEAKER = (ord('-'), ord('_'))
```

Key codes that decrease the push strength: '-' and '_'.

```

```

```

```

```
class Disturbance(Node):
```

The disturbance node class.

```

```

```
    def __init__(self):
```

Constructor.

```
        super().__init__('disturbance')
```

Register the node as `disturbance`.

```

```

```
        self.declare_parameter('world', 'balance_world')
```

`world`: name of the Gazebo world, needed to build the wrench topic names.

```
        self.declare_parameter('link', 'omni_pendulum::pendulum_link')
```

`link`: the scoped Gazebo name of the link to push, `model::link`.

```
        self.declare_parameter('release_delay', 4.0)
```

`release_delay`: seconds after start before the tiny release nudge, giving the robot time to spawn and settle.

```
        self.declare_parameter('release_force', 0.1)
```

`release_force`: 0.1 N, just enough to start the pole falling under gravity.

```
        self.declare_parameter('nudge_duration', 0.15)
```

`nudge_duration`: 0.15 s, roughly the contact time of a finger tap.

```
        self.declare_parameter('min_contact_height', 0.3)
```

`min_contact_height`: lowest point on the rod where a nudge lands (m above the pivot).

```
        self.declare_parameter('max_contact_height', 0.5)
```

`max_contact_height`: highest point (the rod is 0.5 m long).

```
54          self.declare_parameter('key_force', 0.8)
```

`key_force`: starting strength of arrow-key nudges (N).

```
55          self.declare_parameter('key_force_step', 0.2)
```

`key_force_step`: how much one '+' or '-' press changes the strength (N).

```
56
57          world = self.get_parameter('world').value
58          self.link = self.get_parameter('link').value
59          self.release_delay = self.get_parameter('release_delay').value
60          self.release_force = self.get_parameter('release_force').value
61          self.duration = self.get_parameter('nudge_duration').value
62          self.height_range = (self.get_parameter('min_contact_height').value,
63                               self.get_parameter('max_contact_height').value)
64          self.key_force = self.get_parameter('key_force').value
65          self.key_force_step = self.get_parameter('key_force_step').value
```

Lines 57–65 Read all parameters into attributes. Ranges are stored as (min, max) tuples so they can be passed straight to `random.uniform(*range)`.

```
66
67          self.wrench_pub = self.create_publisher(
68              EntityWrench, f'/world/{world}/wrench/persistent', 10)
```

Lines 67–68 Publisher for `/world/balance_world/wrench/persistent`. ros_gz_bridge forwards it to Gazebo's ApplyLinkWrench system, which applies the wrench every physics step until cleared. A persistent wrench is used because a one-shot wrench only lasts a single 1 ms step, too short to shape a 150 ms push.

```
69          self.clear_pub = self.create_publisher(
70              Entity, f'/world/{world}/wrench/clear', 10)
```

Lines 69–70 Publisher for the matching `wrench/clear` topic, used at start-up to remove any leftover wrench and after each nudge to empty Gazebo's stack of wrenches. Without that clearing, each update stayed in the stack for the rest of the run.

```
71          self.create_subscription(Vector3, '/nudge', self.nudge_callback, 10)
```

Subscribe to /nudge for manual pushes (force vector in N).

```
72          self.create_subscription(Int32, '/keyboard/keypress', self.key_callback, 10)
```

Subscribe to /keyboard/keypress for arrow-key pushes and strength changes.

```
73
74          self.start_time = None
```

Simulation time when the node first saw the clock.

```
75          self.released = False
```

Whether the start-up release nudge has happened.

```
76          self.nudge = None           # (start time, peak torque about x, about y)
```

The nudge currently in progress: (start time, peak torque about X, peak torque about Y), or None.

```
77          self.pending = []           # manual nudges waiting for the next tick
```

Queue of manual / keyboard nudges waiting to be applied.

```
78          self.applied = [0.0, 0.0]   # total torque Gazebo currently applies
```

The total torque Gazebo is currently applying. Needed because Gazebo adds persistent wrenches together: to go from torque A to torque B, the node must send B - A.

```
79          self.clear_time = None      # when to clear wrenches after a nudge
```

Simulation time at which to clear Gazebo's wrench stack after a nudge, or None if nothing is due.

```
80
81          self.create_timer(0.01, self.tick)
```

Run `tick` every 0.01 s of simulation time (100 Hz), which samples the 0.15 s half-sine pulse 15 times.

```
82
83      def entity(self):
```

Helper that builds the `Entity` message identifying the pendulum link.

```
84          ent = Entity()
```

New empty Entity message.

```
85          ent.name = self.link
```

Name of the link to push.

```
86          ent.type = Entity.LINK
```

Tell Gazebo the name refers to a link (not a model, joint, etc.).

```
87          return ent
```

Return it.

```
88
89      def nudge_callback(self, msg):
```

Callback for /nudge.

```
90          self.pending.append((msg.x, msg.y))
```

Queue the requested force (x, y) in newtons.

```
91
92      def key_callback(self, msg):
```

Callback for /keyboard/keypress.

```
93          if msg.data in KEY_DIRECTIONS:
```

If the key is an arrow key...

```
94              # Holding a key auto-repeats; don't let presses pile up
```

Comment: holding an arrow key makes the operating system repeat it many times per second.

```
95              if len(self.pending) < 2:
```

Only queue the press if fewer than 2 are waiting, so holding a key cannot stack up dozens of pushes and knock the pole over.

```
96                  dx, dy = KEY_DIRECTIONS[msg.data]
```

Look up the unit direction for this arrow.

```
97                  self.pending.append((dx * self.key_force, dy * self.key_force))
```

Queue a push of `key_force` newtons in that direction.

```
98          elif msg.data in KEY_STRONGER + KEY_WEAKER:
```

Otherwise, if the key is '+', '=', '-' or '_'...

```
99              step = self.key_force_step if msg.data in KEY_STRONGER else -self.key_force_step
```

...choose +step or -step...

```
100             self.key_force = min(max(self.key_force + step, self.key_force_step), 3.0)
```

...and change the strength, clamped between one step (0.2 N) and 3.0 N.

```
101             self.get_logger().info(f'Arrow-key nudge strength: {self.key_force:.1f} N')
```

Log the new strength so you can see it in the launch terminal.

```
102
103     def start_nudge(self, t, fx, fy, label):
```

Starts a nudge with force (fx, fy) newtons at time t.

```
104         height = random.uniform(*self.height_range)
```

Pick a random contact height on the rod.

```
105         # A force along +X at height h tips the pole about +Y, and +Y about -X
```

Comment: how a force becomes a torque.

```
106         self.nudge = (t, -fy * height, fx * height)
```

Torque = height x force. A push along +X at height h rotates the pole about +Y (torque_y = fx*h); a push along +Y rotates it about -X (torque_x = -fy*h). These are the peak torques; the pulse shape scales them over time.

```
107         self.get_logger().info(
108             f'{label}: {math.hypot(fx, fy):.2f} N toward '
109             f'{math.degrees(math.atan2(fy, fx)):.0f} deg at {height:.2f} m up the pole')
```

Lines 107–109 Log the nudge: strength, direction in degrees (0 = forward, 90 = left) and contact height.

```
110
111     def random_force(self, magnitude):
```

Returns a force vector of the given magnitude in a random direction. Only the start-up nudge uses it.

```
112         heading = random.uniform(-math.pi, math.pi)
```

Random direction between -180 and +180 degrees.

```
113         return magnitude * math.cos(heading), magnitude * math.sin(heading)
```

Convert direction and magnitude into x and y components.

```
114
115     def tick(self):
```

Runs every 10 ms: decides when nudges start and sends torque updates.

```
116         t = self.get_clock().now().nanoseconds * 1e-9
```

Current simulation time in seconds.

```
117         if t == 0.0:
```

Before the first /clock message arrives the time reads 0...

```
118             return  # /clock not received yet
```

...so do nothing yet.

```
119         if self.start_time is None:
```

On the first tick with a valid clock...

```
120             self.start_time = t
```

...remember the start time...

```
121             self.clear_pub.publish(self.entity())
```

...clear any persistent wrench left over from a previous run of this node...

```
122             return
```

...and wait for the next tick.

```
123
124         if self.nudge is None:
```

Only start a new nudge when none is in progress.

```
125             if not self.released and t - self.start_time >= self.release_delay:
```

If the release nudge has not happened and its delay has passed...

```
126                 self.released = True
```

...mark it done...

```
127                 self.start_nudge(t, *self.random_force(self.release_force), 'Release nudge')
```

...start a tiny nudge in a random direction...

```
            elif self.pending:
```

Otherwise, if a manual or keyboard nudge is waiting (there are no automatic nudges after the start-up one)...

```
                self.start_nudge(t, *self.pending.pop(0), 'Manual nudge')
```

...start the oldest one.

```

        target = [0.0, 0.0]
```

Default: no torque.

```
        if self.nudge is not None:
```

If a nudge is in progress...

```
            start, tx, ty = self.nudge
```

...unpack its start time and peak torques.

```
            phase = (t - start) / self.duration
```

Progress through the pulse, from 0 (start) to 1 (end).

```
            if phase >= 1.0:
```

If the pulse is over...

```
                self.nudge = None
```

...end the nudge (target stays 0).

```
                self.clear_time = t + 0.1
```

Schedule the clear for 0.1 s after the nudge ends.

```
            else:
```

Otherwise...

```
                shape = math.sin(math.pi * phase)
```

...compute the half-sine shape: 0 at the start, 1 in the middle, 0 at the end. A smooth pulse feels like a finger tap and avoids an instantaneous jolt.

```
                target = [tx * shape, ty * shape]
```

Scale the peak torques by the shape.

```

        delta = [tg - ap for tg, ap in zip(target, self.applied)]
```

The change needed to go from the torque Gazebo applies now to the target torque.

```
        if abs(delta[0]) + abs(delta[1]) > 1e-6:
```

Only publish if it changed noticeably (avoids flooding the bridge with zero updates).

```
            msg = EntityWrench()
```

New wrench message.

```
            msg.entity = self.entity()
```

Which link to apply it to.

```
            msg.wrench.torque.x = delta[0]
```

Torque change about the world X axis (N*m).

```
            msg.wrench.torque.y = delta[1]
```

Torque change about the world Y axis (N*m).

```
            self.wrench_pub.publish(msg)
```

Send it; Gazebo adds it to the persistent wrench.

```
            self.applied = target
```

Remember the new total.

```
150
151         # Wait for the last update to arrive first; clear and wrench are separate topics
```

Comment: the delay exists because wrench updates and clear commands travel on separate topics and could otherwise arrive out of order.

```
152         if self.nudge is None and self.clear_time is not None and t >= self.clear_time:
```

If no nudge is running and a clear is due...

```
153             self.clear_pub.publish(self.entity())
```

...empty Gazebo's stack of wrenches for the pendulum link...

```
154             self.clear_time = None
```

...and mark it done. Tests showed the simulation then holds a steady 1000 joint-state messages per second.

```
155
156
157 def main(args=None):
158     rclpy.init(args=args)
159     node = Disturbance()
160     try:
161         rclpy.spin(node)
162     except KeyboardInterrupt:
163         pass
164     node.destroy_node()
165     rclpy.try_shutdown()
166
167
168 if __name__ == '__main__':
169     main()
```

Lines 157–169 Entry point, identical in structure to the controller's: initialise ROS, create the node, spin until Ctrl+C, then clean up.

keyboard_nudge.py

`src/omni_pendulum_sim/omni_pendulum_sim/keyboard_nudge.py` · 85 lines

An optional terminal program. It reads arrow keys and publishes their key codes on /keyboard/keypress, the same topic as the Gazebo window, so disturbance.py handles both.

```
"""
Nudge the pendulum from a terminal with the arrow keys.

Run in its own terminal while the simulation is running:
    ros2 run omni_pendulum_sim keyboard_nudge

Key presses are published as Qt key codes on /keyboard/keypress, the same
topic Gazebo's Key Publisher plugin uses, and the disturbance node turns them
into nudges.
"""
```

Lines 1–10 Module docstring: how to run the node and how it fits with the rest of the system (it publishes the same key codes as Gazebo's Key Publisher, so the disturbance node handles both).

```

import os
```

`os.read` reads raw bytes from the terminal without Python's buffering.

```
import select
```

`select` waits until the terminal has input, with a timeout, so the loop never blocks forever.

```
import sys
```

`sys.stdin` is the terminal input.

```
import termios
```

`termios` saves and restores terminal settings.

```
import tty
```

`tty.setcbreak` switches the terminal so each key press is delivered immediately (without waiting for Enter) and is not echoed.

```

from omni_pendulum_sim.disturbance import KEY_DOWN, KEY_LEFT, KEY_RIGHT, KEY_UP
```

Reuse the Qt key-code constants from disturbance.py so both files always agree.

```
import rclpy
```

ROS 2 Python client library.

```
from rclpy.node import Node
```

Base class for ROS 2 nodes.

```
from std_msgs.msg import Int32
```

`Int32` message used for key codes.

```


ARROWS = {'A': KEY_UP, 'B': KEY_DOWN, 'C': KEY_RIGHT, 'D': KEY_LEFT}
```

Terminals send arrow keys as the escape sequence ESC [A/B/C/D. This maps the final letter to the Qt code: A = Up, B = Down, C = Right, D = Left.

```
NAMES = {KEY_UP: 'forward +X', KEY_DOWN: 'back -X', KEY_LEFT: 'left +Y', KEY_RIGHT: 'right -Y'}
```

Human-readable direction names for the feedback messages.

```
26
27  HELP = """
28  Nudge the pendulum:
29     Up / Down     push forward / back
30     Left / Right  push left / right
31     + / -         stronger / weaker push
32     q             quit
33  """
```

Lines 27–33 The help text printed when the node starts.

```
34
35
36  def read_key(timeout):
```

Reads one key press, waiting at most `timeout` seconds. Returns a Qt key code for arrows, the character for other keys, or None.

```
37      # Read raw bytes: Python's buffered stdin would swallow the rest of an
38      # arrow key's escape sequence and hide it from select()
```

Lines 37–38 Comment: why raw bytes are read. Python's `sys.stdin.read(1)` reads the whole arrow-key sequence into an internal buffer, so a following `select` would report no more data and the arrow would be lost. This was a real bug found while testing through a pseudo-terminal.

```
39      fd = sys.stdin.fileno()
```

File descriptor (OS-level handle) of the terminal input.

```
40      if not select.select([fd], [], [], timeout)[0]:
```

Wait for input; if none arrives within the timeout...

```
41          return None
```

...return None.

```
42      data = os.read(fd, 8).decode(errors='ignore')
```

Read up to 8 raw bytes and decode them to text.

```
43      if data.startswith('\x1b'):  # arrow keys arrive as ESC [ A..D
```

An arrow key starts with the escape character (byte 27)...

```
44          while len(data) < 3 and select.select([fd], [], [], 0.05)[0]:
```

...if the rest of the sequence has not arrived yet, wait up to 50 ms for it...

```
45              data += os.read(fd, 8).decode(errors='ignore')
```

...and append it.

```
46          return ARROWS.get(data[2]) if data[1:2] == '[' and len(data) >= 3 else None
```

If it is ESC [followed by a letter, return the matching arrow code; otherwise (a lone Esc, other sequences) return None.

```
47      return data[0] if data else None
```

A normal key: return its first character.

```
48
49
50  def main(args=None):
```

Entry point for `ros2 run omni_pendulum_sim keyboard_nudge`.

```
51      if not sys.stdin.isatty():
```

This node needs a real terminal to read keys from...

```
52          print('keyboard_nudge needs an interactive terminal; run it with ros2 run.')
```

...so explain the problem if it is started without one (e.g. from a launch file)...

```
        return
```
...and exit.
```

    rclpy.init(args=args)
```
Initialise ROS 2.
```    node = Node('keyboard_nudge')
```
Create a plain node named `keyboard_nudge` (no subclass needed: it only publishes).
```    pub = node.create_publisher(Int32, '/keyboard/keypress', 10)
```
Publisher for /keyboard/keypress, which the disturbance node listens to.
```    print(HELP)
```
Show the key help.
```

    settings = termios.tcgetattr(sys.stdin)
```
Save the current terminal settings so they can be restored on exit.
```    try:
```
Start a block whose `finally` always restores the terminal.
```        tty.setcbreak(sys.stdin.fileno())
```
Switch to cbreak mode: keys arrive instantly, without Enter.
```        while rclpy.ok():
```
Loop until ROS shuts down.
```            key = read_key(0.1)
```
Wait up to 0.1 s for a key.
```            if key is None:
```
No key pressed...
```                continue
```
...check again.
```            if key in ('q', '\x03'):
```
'q' or Ctrl+C (character 3 in cbreak mode)...
```                break
```
...leaves the loop.
```            code = key if isinstance(key, int) else ord(key)
```
Arrow keys are already integer codes; other keys are converted to their character code.
```            if code in NAMES or key in '+=-_':
```
Only arrows and the strength keys are sent.
```                pub.publish(Int32(data=code))
```
Publish the key code.
```                if code in NAMES:
```
If it was an arrow...
```                    print(f'Nudge {NAMES[code]}', flush=True)
```
...print which way the push goes.
```                else:
```
Otherwise it was a strength key...
```                    print('Stronger push' if key in '+=' else 'Weaker push', flush=True)
```
...print whether the push got stronger or weaker.

```
76      except KeyboardInterrupt:
```

Ctrl+C can also arrive as an exception...

```
77          pass
```

...which is fine.

```
78      finally:
```

Always run this cleanup, even after an error.

```
79          termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
```

Restore the original terminal settings; otherwise the terminal would stay in cbreak mode after the program exits.

```
80          node.destroy_node()
```

Release the node.

```
81          rclpy.try_shutdown()
```

Shut ROS down if it is still running.

```
82
83
84  if __name__ == '__main__':
85      main()
```

Lines 84–85 Run `main` only when the file is executed directly.

sim.launch.py

`src/omni_pendulum_sim/launch/sim.launch.py` · 113 lines

One command to start everything. It only describes what to run; the launch system then starts each process: Gazebo, the bridge, the nodes and, after 2 s, the robot spawner.

```
1  import os
```

`os.path.join` builds file paths that work on any system.

```
2
3  from ament_index_python.packages import get_package_share_directory
```

Finds where a ROS package's shared files (launch, urdf, meshes, worlds, config) were installed, e.g. `install/omni_pendulum_sim/share/omni_pendulum_sim`.

```
4  from launch import LaunchDescription
```

`LaunchDescription` is the list of everything this launch file starts.

```
5  from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, TimerAction
```

`DeclareLaunchArgument` creates command-line options such as `gui:=false`; `IncludeLaunchDescription` runs another launch file; `TimerAction` delays an action.

```
6  from launch.conditions import IfCondition, UnlessCondition
```

`IfCondition` / `UnlessCondition` start an action only when a launch argument is true / false.

```
7  from launch.launch_description_sources import PythonLaunchDescriptionSource
```

Tells `IncludeLaunchDescription` that the included file is a Python launch file.

```
8  from launch.substitutions import Command, LaunchConfiguration
```

`Command` runs a shell command at launch time and uses its output; `LaunchConfiguration` reads the value of a launch argument.

```
9  from launch_ros.actions import Node
```

`Node` describes a ROS 2 program to start.

```
10 from launch_ros.parameter_descriptions import ParameterValue
```

`ParameterValue` wraps a value and its type, so the URDF text from xacro is passed as a string parameter (not parsed as YAML).

```
11
12 WORLD_NAME = 'balance_world'
```

Name of the Gazebo world. It must match `<world name=...>` in balance_world.sdf, because the spawn service and the wrench topics contain it.

```
13
14
15 def generate_launch_description():
```

ROS 2 calls this function when you run `ros2 launch omni_pendulum_sim sim.launch.py`.

```
16     pkg_omni_pendulum = get_package_share_directory('omni_pendulum_sim')
```

Install location of this package's shared files.

```
17     pkg_ros_gz_sim = get_package_share_directory('ros_gz_sim')
```

Install location of ros_gz_sim, which provides the standard Gazebo launch file and the `create` spawner.

```
18
19     xacro_file = os.path.join(pkg_omni_pendulum, 'urdf', 'robot.urdf.xacro')
```

Path to the robot description (xacro = XML macros that expand into URDF).

```
20     world = os.path.join(pkg_omni_pendulum, 'worlds', f'{WORLD_NAME}.sdf')
```

Path to the world file (ground, light, physics settings and the ApplyLinkWrench system).

```
21      gui_config = os.path.join(pkg_omni_pendulum, 'config', 'gui.config')
```

Path to the custom Gazebo GUI layout (default layout + Key Publisher + closer camera).

```
22      robot_description = ParameterValue(Command(['xacro ', xacro_file]), value_type=str)
```

Run `xacro robot.urdf.xacro` at launch time; its output, the full URDF text, becomes the `robot_description` parameter. Doing this at launch time means edits to the xacro take effect on the next launch without rebuilding.

```
23
24      gui = LaunchConfiguration('gui')
```

Reads the `gui` launch argument.

```
25      push = LaunchConfiguration('push')
```

Reads the `push` launch argument (disturbance node on/off).

```
26      balance = LaunchConfiguration('balance')
```

Reads the `balance` launch argument (controller on/off).

```
27      declare_gui = DeclareLaunchArgument(
28          'gui', default_value='true', description='Start the Gazebo GUI')
```

Lines 27–28 Declares `gui` (default true). `gui:=false` runs Gazebo without a window, which was used for all the automated tests.

```
29      declare_push = DeclareLaunchArgument(
30          'push', default_value='true', description='Start-up nudge and manual nudges')
```

Lines 29–30 Declares `push` (default true). `push:=false` does not start the disturbance node: no start-up nudge and no manual nudges, so the pole just stands upright and the robot never reacts.

```
31      declare_balance = DeclareLaunchArgument(
32          'balance', default_value='true',
33          description='Run the balance controller (false = watch the pole fall)')
```

Lines 31–33 Declares `balance` (default true). `balance:=false` lets you watch the pole fall with no controller, which proves gravity and the free joints work.

```
34
35      node_robot_state_publisher = Node(
```

robot_state_publisher reads the URDF and /joint_states and publishes the TF transform of every link, which tools such as RViz use to display the robot.

```
36          package='robot_state_publisher',
```

The ROS package containing the program.

```
37          executable='robot_state_publisher',
```

The program (executable) name inside that package.

```
38          output='screen',
```

Print its output in the launch terminal.

```
39          parameters=[{'robot_description': robot_description, 'use_sim_time': True}]
```

Give it the URDF text, and tell it to use simulation time from /clock so its transforms carry Gazebo timestamps.

```
40      )
```

End of the Node description.

```
41
42      # Gazebo server and GUI in one process, so the GUI always shows this world;
43      # gui:=false runs the server only (-s)
```

Lines 42–43 Comment. The server (physics) and GUI (window) used to be started as two separate processes. When an old server was left running in the background, the new GUI attached to the wrong one and the robot was missing from the entity tree. Starting both in one process guarantees the window shows this simulation.

```
    gz_sim_launch = os.path.join(pkg_ros_gz_sim, 'launch', 'gz_sim.launch.py')
```

Path to ros_gz_sim's standard launch file for Gazebo.

```
    gz_sim = IncludeLaunchDescription(
```

Include that launch file to start Gazebo with the GUI...

```
        PythonLaunchDescriptionSource(gz_sim_launch),
```

...it is a Python launch file...

```
        launch_arguments={'gz_args': f'-r -v 3 {world} --gui-config {gui_config}',
                          'on_exit_shutdown': 'true'}.items(),
```

Lines 47–48 ...with these Gazebo arguments: `-r` starts the simulation running (not paused), `-v 3` sets log verbosity, then the world file and `--gui-config` with our layout. `on_exit_shutdown` stops the whole launch when the Gazebo window is closed.

```
        condition=IfCondition(gui)
```

...only when `gui` is true.

```
    )
```

End of the include.

```
    gz_server = IncludeLaunchDescription(
```

A second include of the same launch file for headless mode...

```
        PythonLaunchDescriptionSource(gz_sim_launch),
```

...Python launch file...

```
        launch_arguments={'gz_args': f'-r -s -v 3 {world}', 'on_exit_shutdown': 'true'}.items
(),
```

...with `-s` (server only, no window).

```
        condition=UnlessCondition(gui)
```

...only when `gui` is false.

```
    )
```

End of the include.

```

    # base_footprint is at ground level; spawn just above it so the landing
    # does not knock the pendulum over
```

Lines 57–58 Comment: why the spawn height is tiny.

```
    node_spawn_entity = Node(
```

The `create` program from ros_gz_sim asks Gazebo to add the robot to the world.

```
        package='ros_gz_sim',
```

Package containing `create`.

```
        executable='create',
```

Executable name.

```
        output='screen',
```

Show its output.

```
        arguments=['-world', WORLD_NAME, '-topic', 'robot_description',
                   '-name', 'omni_pendulum', '-z', '0.005']
```

Lines 63–64 Arguments: spawn into `balance_world`, read the robot description from the `robot_description` topic (published by robot_state_publisher), name the model `omni_pendulum` (the disturbance node refers to `omni_pendulum::pendulum_link`), and place it 5 mm above the ground so it settles gently instead of dropping and knocking the pole over.

```
    )
```

End of the spawner Node.

```
66
67      node_ros_gz_bridge = Node(
```

ros_gz_bridge's `parameter_bridge` translates messages between ROS 2 (DDS) and Gazebo Transport, two separate communication systems.

```
68          package='ros_gz_bridge',
```

Package of the bridge.

```
69          executable='parameter_bridge',
```

Executable name.

```
70          arguments=[
```

Each argument is `topic@ROS_type<direction>Gazebo_type`. `]` means ROS to Gazebo, `[` means Gazebo to ROS.

```
71              '/cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist',
```

Velocity commands from the controller go to Gazebo's MecanumDrive system.

```
72              '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
```

Joint states from Gazebo come to ROS (Gazebo sends them as a `Model` message; the bridge converts them to `JointState`).

```
73              '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
```

Wheel-based odometry from MecanumDrive comes to ROS.

```
74              '/ground_truth/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
```

Ground-truth odometry (true pose and velocity from the physics engine) comes to ROS. Added to measure wheel slip.

```
75              '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
```

The simulation clock comes to ROS. Every node with `use_sim_time: True` uses it instead of the computer's clock.

```
76              '/keyboard/keypress@std_msgs/msg/Int32[gz.msgs.Int32',
```

Key presses from the Gazebo window's Key Publisher plugin come to ROS for the disturbance node.

```
77              f'/world/{WORLD_NAME}/wrench/persistent'
78              '@ros_gz_interfaces/msg/EntityWrench]gz.msgs.EntityWrench',
```

Lines 77–78 Persistent wrenches (nudges) go from ROS to Gazebo's ApplyLinkWrench system. The f-string and the plain string on the next line are joined by Python into one argument.

```
79              f'/world/{WORLD_NAME}/wrench/clear@ros_gz_interfaces/msg/Entity]gz.msgs.Entity',
```

Wrench-clear commands go from ROS to Gazebo.

```
80          ],
```

End of the argument list.

```
81          parameters=[{'use_sim_time': True}],
```

The bridge stamps messages using simulation time.

```
82          output='screen'
```

Show its output.

```
83      )
```

End of the bridge Node.

```
84
85      node_balance_controller = Node(
```

The balance controller node from this package.

```
86          package='omni_pendulum_sim',
```

This package.

```
87          executable='balance_controller',
```

The `balance_controller` executable (defined in setup.py).

```
88          parameters=[{'use_sim_time': True}],
```

Use simulation time, so the controller's clock matches the physics.

```
89          output='screen',
```

Show its log messages.

```
90          condition=IfCondition(balance)
```

Start it only when `balance` is true.

```
91      )
```

End of the controller Node.

```
92
93      node_disturbance = Node(
```

The disturbance (nudge) node.

```
94          package='omni_pendulum_sim',
```

This package.

```
95          executable='disturbance',
```

The `disturbance` executable.

```
96          parameters=[{'use_sim_time': True, 'world': WORLD_NAME}],
```

Simulation time, and the world name for the wrench topics.

```
97          output='screen',
```

Show its log messages (each nudge is printed).

```
98          condition=IfCondition(push)
```

Start it only when `push` is true.

```
99      )
```

End of the disturbance Node.

```
100
101     return LaunchDescription([
```

Return everything to start, in order.

```
102         declare_gui,
103         declare_push,
104         declare_balance,
```

Lines 102–104 The three launch arguments must be declared first.

```
105         node_robot_state_publisher,
```

robot_state_publisher (also publishes `robot_description`, which the spawner needs).

```
106         gz_sim,
```

Gazebo with GUI (if gui:=true).

```
107         gz_server,
```

Gazebo headless (if gui:=false). Exactly one of the two starts.

```
108         node_ros_gz_bridge,
```

The bridge.

```
109         node_balance_controller,
```

The controller.

```
110         node_disturbance,
```

The disturbance node.

```
111         # Give Gazebo and robot_state_publisher a moment before spawning
```

Comment.

```
112         TimerAction(period=2.0, actions=[node_spawn_entity]),
```

Spawn the robot after 2 seconds, giving Gazebo time to load the world and robot_state_publisher time to publish the description.

```
113     ])
```

End of the list and function.

robot.urdf.xacro

`src/omni_pendulum_sim/urdf/robot.urdf.xacro` · 275 lines

The robot model. It lists links (rigid bodies) and joints (connections), with properties at the top and macros for repeated parts. Gazebo-only settings sit inside `<gazebo>` blocks.

```
<?xml version="1.0"?>
```

XML declaration: this file is XML.

```
<!--
  AgileX LIMO in mecanum (omnidirectional) mode carrying a 2-DOF inverted pendulum.

  Chassis/wheel dimensions and meshes come from AgileX's limo_description package
  (https://github.com/agilexrobotics/limo_ros2). That package only ships diff-drive and
  Ackermann Gazebo Classic models, so the mecanum behaviour is added here for Gazebo
  Harmonic with anisotropic wheel friction + the gz MecanumDrive system.
-->
```

Lines 2–9 Header comment: what the model is and where the LIMO dimensions and meshes came from, and that the mecanum behaviour was added for Gazebo Harmonic because AgileX's package only has diff-drive and Ackermann models for the old Gazebo Classic.

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" xmlns:gz="http://gazebosim.org/schema" name
="omni_pendulum">
```

The root element. `xmlns:xacro` enables xacro macros/properties; `xmlns:gz` declares the Gazebo namespace needed for the `gz:expressed_in` attribute on the wheel friction (without it xacro fails with 'unbound prefix'). `name` is the robot's name.

```

  <xacro:property name="M_PI" value="3.14159265"/>
```

xacro property (a named constant) for pi.

```
  <xacro:property name="mesh_dir" value="file://$(find omni_pendulum_sim)/meshes/limo"/>
```

Folder of the LIMO meshes. `$(find omni_pendulum_sim)` is replaced by the package's install path when xacro runs; `file://` makes it an absolute path both Gazebo and RViz can load.

```

  <!-- LIMO geometry (from limo_four_diff.xacro) -->
```

Comment: the next values come from AgileX's limo_four_diff.xacro.

```
  <xacro:property name="base_x_size" value="0.13"/>
```

Chassis collision box length (m).

```
  <xacro:property name="base_y_size" value="0.12"/>
```

Chassis collision box width (m).

```
  <xacro:property name="base_z_size" value="0.1"/>
```

Original box height, still used for the chassis inertia formula.

```
  <xacro:property name="base_mass" value="2.1557"/>
```

Chassis mass (kg), from AgileX's URDF.

```
  <xacro:property name="base_height" value="0.15"/>
```

Height of base_link above the ground (m).

```
  <xacro:property name="wheelbase" value="0.2"/>
```

Distance between front and rear axles (m).

```
  <xacro:property name="track" value="0.13"/>
```

Distance between the left and right wheel mounting points (m).

```
23   <xacro:property name="wheel_vertical_offset" value="-0.10"/>
```
Wheel axle height relative to base_link (m, below it).
```
24   <xacro:property name="wheel_length" value="0.045"/>
```
Wheel width (m).
```
25   <xacro:property name="wheel_radius" value="0.045"/>
```
Wheel radius (m).
```
26   <xacro:property name="wheel_mass" value="0.5"/>
```
Mass of each wheel (kg).
```
27   <!-- Height of the LIMO's top surface at the centre of the body, measured from the mesh -->
```
Comment.
```
28   <xacro:property name="body_top_height" value="0.1455"/>
```
Height of the LIMO's top surface at the centre, measured from the mesh's vertices (0.1455 m). The body is not flat: the centre is a dip and a sloped cover peaks at 0.2535 m.
```
29
30   <!-- Flat platform on four posts, clearing the LIMO's sloped top cover (peak 0.2535 m) -->
```
Comment: the platform was added so the pole sits on a flat surface above the sloped cover instead of floating.
```
31   <xacro:property name="platform_top_height" value="0.266"/>
```
Height of the platform's top surface above the ground (just above the cover's highest point).
```
32   <xacro:property name="platform_x_size" value="0.26"/>
```
Platform length (m).
```
33   <xacro:property name="platform_y_size" value="0.17"/>
```
Platform width (m).
```
34   <xacro:property name="platform_thickness" value="0.008"/>
```
Platform plate thickness (m).
```
35   <xacro:property name="platform_mass" value="0.2"/>
```
Platform mass (kg).
```
36   <xacro:property name="post_x" value="0.10"/>
```
X position of the support posts (m).
```
37   <xacro:property name="post_y" value="0.065"/>
```
Y position of the support posts (m).
```
38   <xacro:property name="post_radius" value="0.006"/>
```
Post radius (m).
```
39   <xacro:property name="post_bottom_height" value="0.12"/>
```
Height where the posts start (they disappear into the body).
```
40
41   <!-- Pendulum: thin rod with a ball on top -->
```
Comment.
```
42   <xacro:property name="mount_thickness" value="0.012"/>
```
Thickness of the small puck under the pivot (m).
```
43   <xacro:property name="rod_length" value="0.5"/>
```
Rod length (m).
```
44   <xacro:property name="rod_radius" value="0.008"/>
```
Rod radius (m).

```
45   <xacro:property name="rod_mass" value="0.1"/>
```

Rod mass (kg).

```
46   <xacro:property name="bob_radius" value="0.025"/>
```

Radius of the ball on top (m).

```
47   <xacro:property name="bob_mass" value="0.1"/>
```

Mass of the ball (kg).

```
48
49   <link name="base_footprint"/>
```

`base_footprint` : an empty link at ground level under the robot, the conventional root frame for mobile robots.

```
50
51   <joint name="base_joint" type="fixed">
```

A fixed joint rigidly attaches two links.

```
52     <parent link="base_footprint"/>
```

Parent link.

```
53     <child link="base_link"/>
```

Child link.

```
54     <origin xyz="0 0 ${base_height}" rpy="0 0 0"/>
```

The chassis frame sits 0.15 m above the footprint.

```
55   </joint>
```

End of joint.

```
56
57   <link name="base_link">
```

`base_link` : the LIMO chassis.

```
58     <inertial>
```

Mass properties used by the physics engine.

```
59       <mass value="${base_mass}"/>
```

Chassis mass.

```
60       <inertia ixx="${base_mass * (base_y_size**2 + base_z_size**2) / 12}" ixy="0" ixz="0"
61                iyy="${base_mass * (base_x_size**2 + base_z_size**2) / 12}" iyz="0"
62                izz="${base_mass * (base_x_size**2 + base_y_size**2) / 12}"/>
```

Lines 60–62 Inertia tensor of a solid box: I = m(a^2 + b^2)/12 for each axis. The AgileX file had unrealistic values (0.24 to 0.96 kg*m^2), so they are computed from the box size instead.

```
63     </inertial>
```

End of inertial.

```
64     <visual>
```

What you see (does not affect physics).

```
65       <origin xyz="0 0 ${-base_height}" rpy="0 0 ${M_PI / 2}"/>
```

The mesh's origin is at ground level and it is modelled along a different axis, so it is moved down 0.15 m and rotated 90 degrees about Z to line up with base_link.

```
66       <geometry><mesh filename="${mesh_dir}/limo_base_lite.dae"/></geometry>
```

The simplified LIMO body mesh (41k triangles instead of 756k, so Gazebo no longer freezes while loading).

```
67     </visual>
```

End of visual.

```
68     <!-- Box sits flush with the body's top surface so it never touches the pendulum -->
```

Comment.

```
69     <collision>
```

The shape used for collisions (a simple box, much cheaper than the mesh).

```
70       <origin xyz="0 0 ${body_top_height - base_height - 0.04}"/>
```

The box is lowered so its top is flush with the body's top surface. AgileX's box stuck 5 cm above the real body and would have hit the pendulum.

```
71       <geometry><box size="${base_x_size} ${base_y_size} 0.08"/></geometry>
```

Box size: original length and width, 8 cm tall.

```
72     </collision>
```

End of collision.

```
73   </link>
```

End of base_link.

```
74
75   <!-- Mecanum wheel. Real mecanum rollers are approximated with anisotropic friction:
76        full grip (mu) along fdir1, zero grip (mu2) perpendicular to it. fdir1 is
77        expressed in base_footprint (the root link) so it does not spin with the wheel. -->
```

Lines 75–77 Comment describing the mecanum approximation. Real mecanum wheels have angled rollers; simulating dozens of tiny rollers is slow and unstable. Instead each wheel is a plain cylinder whose friction is anisotropic: full grip along one diagonal direction (fdir1) and zero grip across it, which is the net effect of the rollers.

```
78   <xacro:macro name="mecanum_wheel" params="prefix x_reflect y_reflect fdir">
```

A xacro macro is a reusable template; this one creates one complete wheel. Parameters: name prefix, front/rear sign, left/right sign, friction direction.

```
79     <link name="${prefix}_wheel_link">
```

The wheel link.

```
80       <inertial>
```

Mass properties.

```
81         <origin xyz="0 ${wheel_length / 2} 0"/>
```

Centre of mass in the middle of the wheel's width (the joint is at its inner face).

```
82         <mass value="${wheel_mass}"/>
```

Wheel mass.

```
83         <inertia ixx="${wheel_mass * (3 * wheel_radius**2 + wheel_length**2) / 12}" ixy="0" ixz
   ="0"
84                  iyy="${wheel_mass * wheel_radius**2 / 2}" iyz="0"
85                  izz="${wheel_mass * (3 * wheel_radius**2 + wheel_length**2) / 12}"/>
```

Lines 83–85 Inertia of a solid cylinder whose axis is the link's Y axis: $m*r^2/2$ about the axle, $m(3r^2 + h^2)/12$ about the other two axes.

```
86       </inertial>
```

End of inertial.

```
87       <visual>
```

Visual.

```
88         <geometry><mesh filename="${mesh_dir}/limo_wheel_lite.dae"/></geometry>
```

Simplified LIMO wheel mesh.

```
89       </visual>
```

End of visual.

```
90      <collision>
```
Collision.
```
91        <origin xyz="0 ${wheel_length / 2} 0" rpy="${M_PI / 2} 0 0"/>
```
Shift the cylinder outward by half its width and rotate it 90 degrees so its axis lies along the axle.
```
92        <geometry><cylinder radius="${wheel_radius}" length="${wheel_length}"/></geometry>
```
Cylinder size.
```
93      </collision>
```
End of collision.
```
94    </link>
```
End of wheel link.
```
95
96    <!-- Right wheels are flipped so the mesh faces outward; the axis flips with
97         them, so positive joint velocity always drives the robot forward -->
```
Lines 96–97 Comment explaining the joint orientation trick below.
```
98    <joint name="${prefix}_wheel_joint" type="continuous">
```
A continuous joint spins without limits (a wheel axle).
```
99      <parent link="base_link"/>
```
Attached to the chassis.
```
100     <child link="${prefix}_wheel_link"/>
```
Child: this wheel.
```
101     <origin xyz="${x_reflect * wheelbase / 2} ${y_reflect * track / 2} ${wheel_vertical_offset}"
```
Wheel position: half the wheelbase forward or back, half the track left or right, below the chassis.
```
102             rpy="${(1 - y_reflect) / 2 * M_PI} 0 0"/>
```
Right wheels (y_reflect = -1) are rotated 180 degrees about X so the mesh faces outward; left wheels are not rotated.
```
103     <axis xyz="0 ${y_reflect} 0"/>
```
The spin axis. For right wheels it is flipped too, so after the 180-degree rotation every wheel's positive spin drives the robot forward, which is what MecanumDrive expects.
```
104   </joint>
```
End of joint.
```
105
106   <gazebo reference="${prefix}_wheel_link">
```
Gazebo-specific settings for this wheel link (URDF itself has no friction model).
```
107     <collision>
```
These settings are merged into the wheel's collision when URDF is converted to SDF.
```
108       <surface>
```
Contact surface properties.
```
109         <friction>
```
Friction.
```
110           <ode>
```
ODE-style friction parameters (also understood by Gazebo Harmonic's DART physics).
```
111             <mu>1.0</mu>
```
`mu` = 1.0: full friction along the primary direction fdir1.

```
112                <mu2>0.0</mu2>
```

`mu2` = 0.0: no friction perpendicular to it, so the wheel slides freely sideways along its rollers.

```
113                <fdir1 gz:expressed_in="base_footprint">${fdir}</fdir1>
```

The primary friction direction, a diagonal such as (1, -1, 0). `gz:expressed_in` makes it fixed in base_footprint; otherwise it would rotate with the spinning wheel. base_footprint is used because base_link is merged into it during URDF-to-SDF conversion (fixed joints are 'lumped').

```
114              </ode>
115            </friction>
116          </surface>
117        </collision>
118      </gazebo>
```

Lines 114–118 Closing tags.

```
119    </xacro:macro>
```

End of the macro.

```
120
121    <xacro:mecanum_wheel prefix="front_left" x_reflect="1" y_reflect="1" fdir="1 -1 0"/>
122    <xacro:mecanum_wheel prefix="front_right" x_reflect="1" y_reflect="-1" fdir="1 1 0"/>
123    <xacro:mecanum_wheel prefix="rear_left" x_reflect="-1" y_reflect="1" fdir="1 1 0"/>
124    <xacro:mecanum_wheel prefix="rear_right" x_reflect="-1" y_reflect="-1" fdir="1 -1 0"/>
```

Lines 121–124 Create the four wheels. The friction diagonals form an X pattern when seen from above, like real mecanum rollers; this is what lets the robot strafe sideways. These directions were verified in simulation by driving forward, sideways and diagonally.

```
125
126    <!-- Platform: origin at the centre of the plate's top surface -->
```

Comment.

```
127    <xacro:property name="post_length" value="${platform_top_height - platform_thickness - post_b
ottom_height}"/>
```

Post length: from their starting height up to the underside of the plate.

```
128    <xacro:macro name="platform_post" params="x y">
```

Macro producing one post visual at (x, y).

```
129      <visual>
```

Visual only (the posts have no collision).

```
130        <origin xyz="${x} ${y} ${-platform_thickness - post_length / 2}"/>
```

Centre the post halfway between its bottom and the underside of the plate.

```
131        <geometry><cylinder radius="${post_radius}" length="${post_length}"/></geometry>
```

Post shape.

```
132        <material name="post_mat"><color rgba="0.7 0.7 0.72 1.0"/></material>
```

Grey colour.

```
133      </visual>
```

End of visual.

```
134    </xacro:macro>
```

End of macro.

```
135
136    <link name="platform_link">
```

The platform link.

```
137      <inertial>
```

Mass properties.

```
      <origin xyz="0 0 ${-platform_thickness / 2}"/>
```
Centre of mass in the middle of the plate.
```
      <mass value="${platform_mass}"/>
```
Platform mass.
```
      <inertia ixx="${platform_mass * platform_y_size**2 / 12}" ixy="0" ixz="0"
               iyy="${platform_mass * platform_x_size**2 / 12}" iyz="0"
               izz="${platform_mass * (platform_x_size**2 + platform_y_size**2) / 12}"/>
```
Lines 140–142 Inertia of a thin plate.
```
    </inertial>
```
End of inertial.
```
    <visual>
```
Plate visual.
```
      <origin xyz="0 0 ${-platform_thickness / 2}"/>
```
Centred half a thickness below the link origin (the origin is the plate's top surface).
```
      <geometry><box size="${platform_x_size} ${platform_y_size} ${platform_thickness}"/></geometry>
```
Box shape of the plate.
```
      <material name="platform_mat"><color rgba="0.95 0.75 0.1 1.0"/></material>
```
Yellow colour.
```
    </visual>
```
End of visual.
```
    <collision>
```
The plate has collision, so it physically supports the pendulum mount.
```
      <origin xyz="0 0 ${-platform_thickness / 2}"/>
```
Same position.
```
      <geometry><box size="${platform_x_size} ${platform_y_size} ${platform_thickness}"/></geometry>
```
Same box.
```
    </collision>
```
End of collision.
```
    <xacro:platform_post x="${post_x}" y="${post_y}"/>
    <xacro:platform_post x="${post_x}" y="${-post_y}"/>
    <xacro:platform_post x="${-post_x}" y="${post_y}"/>
    <xacro:platform_post x="${-post_x}" y="${-post_y}"/>
```
Lines 153–156 Four posts at the corners.
```
  </link>
```
End of platform link.
```

```
```
  <joint name="platform_joint" type="fixed">
```
Fixed joint attaching the platform to the chassis.
```
    <parent link="base_link"/>
```
Parent: chassis.
```
    <child link="platform_link"/>
```
Child: platform.
```
    <origin xyz="0 0 ${platform_top_height - base_height}"/>
```
Platform top at 0.266 m above the ground (base_link is at 0.15 m).

```
  </joint>
```
End of joint.
```

```
```
  <!-- Pendulum mount: a small puck on the platform; the pivot is on top of it -->
```
Comment.
```
  <link name="pendulum_mount">
```
The puck under the pivot.
```
    <inertial>
```
Mass properties (small).
```
      <mass value="0.05"/>
```
50 g.
```
      <inertia ixx="0.00001" ixy="0" ixz="0" iyy="0.00001" iyz="0" izz="0.00001"/>
```
Tiny inertia.
```
    </inertial>
```
End of inertial.
```
    <visual>
```
Visual.
```
      <origin xyz="0 0 ${-mount_thickness / 2}"/>
```
Centred half its thickness below the pivot.
```
      <geometry><cylinder radius="0.02" length="${mount_thickness}"/></geometry>
```
Small cylinder.
```
      <material name="mount_mat"><color rgba="0.2 0.2 0.2 1.0"/></material>
```
Dark grey.
```
    </visual>
```
End of visual.
```
  </link>
```
End of link.
```

```
```
  <joint name="pendulum_mount_joint" type="fixed">
```
Fixed joint putting the puck on the platform.
```
    <parent link="platform_link"/>
```
Parent: platform.
```
    <child link="pendulum_mount"/>
```
Child: puck.
```
    <origin xyz="0 0 ${mount_thickness}"/>
```
The puck's origin (the pivot point) is one puck-thickness above the plate.
```
  </joint>
```
End of joint.
```

```
```
  <!-- Pitch Joint: positive angle tips the pendulum toward +X -->
```
Comment.
```
  <joint name="pendulum_pitch_joint" type="revolute">
```
Pitch joint: a revolute (hinge) joint, the first of two hinges that let the pole tip in any direction.
```
    <parent link="pendulum_mount"/>
```
Parent: the puck.

```
187     <child link="dummy_pitch_link"/>
```
Child: an intermediate link.
```
188     <origin xyz="0 0 0" rpy="0 0 0"/>
```
At the pivot point.
```
189     <axis xyz="0 1 0"/>
```
Rotates about Y, so the pole tips forward/back (positive = toward +X by the right-hand rule).
```
190     <limit lower="-1.57" upper="1.57" effort="10.0" velocity="10.0"/>
```
Limits of plus/minus 90 degrees, so a fallen pole rests on its side instead of passing through the robot. Effort and velocity limits are required by URDF.
```
191     <dynamics damping="0.0001" friction="0.0"/>
```
Almost no damping and no friction: the hinge is essentially free, so only gravity, nudges and the robot's motion move the pole.
```
192   </joint>
```
End of joint.
```
193
194   <link name="dummy_pitch_link">
```
Intermediate link between the two hinges. Two perpendicular hinges in series make a universal joint (2 degrees of freedom).
```
195     <inertial>
```
Every link that moves needs mass properties...
```
196       <mass value="0.001"/>
```
...1 g...
```
197       <inertia ixx="0.00001" ixy="0" ixz="0" iyy="0.00001" iyz="0" izz="0.00001"/>
```
...and tiny inertia, so it does not affect the dynamics.
```
198     </inertial>
```
End of inertial.
```
199   </link>
```
End of link.
```
200
201   <!-- Roll Joint: positive angle tips the pendulum toward -Y -->
```
Comment.
```
202   <joint name="pendulum_roll_joint" type="revolute">
```
Roll joint: the second hinge.
```
203     <parent link="dummy_pitch_link"/>
```
Parent: the intermediate link.
```
204     <child link="pendulum_link"/>
```
Child: the pendulum.
```
205     <origin xyz="0 0 0" rpy="0 0 0"/>
```
Same pivot point.
```
206     <axis xyz="1 0 0"/>
```
Rotates about X, so the pole tips left/right (positive = toward -Y by the right-hand rule).
```
207     <limit lower="-1.57" upper="1.57" effort="10.0" velocity="10.0"/>
```
Same limits as pitch.
```
208     <dynamics damping="0.0001" friction="0.0"/>
```
Same near-zero damping.

```
209   </joint>
```

End of joint.

```
210
211   <!-- Rod + ball combined into one rigid link; inertia is about the combined COM -->
```

Comment.

```
212   <xacro:property name="pend_mass" value="${rod_mass + bob_mass}"/>
```

Total pendulum mass: 0.2 kg.

```
213   <xacro:property name="pend_com" value="${(rod_mass * rod_length / 2 + bob_mass * rod_length)
      / pend_mass}"/>
```

Combined centre of mass height: (0.1*0.25 + 0.1*0.5)/0.2 = 0.375 m above the pivot.

```
214   <xacro:property name="pend_ixx" value="${rod_mass * rod_length**2 / 3 + bob_mass * rod_length
      **2
215                                          + 0.4 * bob_mass * bob_radius**2 - pend_mass * pend_c
      om**2}"/>
```

Lines 214–215 Inertia about the combined centre of mass: rod about its end ($mL^2/3$) plus ball about the pivot ($mL^2 + 2mr^2/5$), minus total mass times COM height squared (parallel-axis theorem in reverse). Getting this right matters because the controller gains were computed from the same numbers ($J = 0.0334\ kg*m^2$ about the pivot).

```
216   <link name="pendulum_link">
```

The pendulum link (rod + ball as one rigid body).

```
217     <inertial>
```

Mass properties.

```
218       <origin xyz="0 0 ${pend_com}"/>
```

Centre of mass position.

```
219       <mass value="${pend_mass}"/>
```

Total mass.

```
220       <inertia ixx="${pend_ixx}" ixy="0" ixz="0" iyy="${pend_ixx}" iyz="0"
221                izz="${rod_mass * rod_radius**2 / 2 + 0.4 * bob_mass * bob_radius**2}"/>
```

Lines 220–221 Inertia tensor: equal about X and Y (symmetric pole), very small about the pole's own axis (Z).

```
222     </inertial>
```

End of inertial.

```
223     <visual>
```

Rod visual.

```
224       <origin xyz="0 0 ${rod_length / 2}"/>
```

Centred halfway up the rod.

```
225       <geometry><cylinder radius="${rod_radius}" length="${rod_length}"/></geometry>
```

Cylinder.

```
226       <material name="rod_mat"><color rgba="0.9 0.1 0.1 1.0"/></material>
```

Red.

```
227     </visual>
```

End.

```
228     <visual>
```

Ball visual.

```
229       <origin xyz="0 0 ${rod_length}"/>
```

At the top of the rod.

```
230       <geometry><sphere radius="${bob_radius}"/></geometry>
```

Sphere.

```
231       <material name="bob_mat"><color rgba="0.1 0.3 0.9 1.0"/></material>
```

Blue.

```
232     </visual>
```

End.

```
233     <collision>
234       <origin xyz="0 0 ${rod_length / 2}"/>
235       <geometry><cylinder radius="${rod_radius}" length="${rod_length}"/></geometry>
236     </collision>
237     <collision>
238       <origin xyz="0 0 ${rod_length}"/>
239       <geometry><sphere radius="${bob_radius}"/></geometry>
240     </collision>
```

Lines 233–240 Collision shapes matching the visuals, so the pole can hit the ground or the robot when it falls.

```
241   </link>
```

End of pendulum link.

```
242
243   <!-- Gazebo Plugins -->
```

Comment.

```
244   <gazebo>
```

A `<gazebo>` block with no reference applies to the whole model: it holds Gazebo system plugins.

```
245     <plugin filename="gz-sim-mecanum-drive-system" name="gz::sim::systems::MecanumDrive">
```

MecanumDrive: turns a body velocity command (/cmd_vel) into the four wheel speeds using mecanum inverse kinematics, and publishes wheel odometry.

```
246       <front_left_joint>front_left_wheel_joint</front_left_joint>
247       <front_right_joint>front_right_wheel_joint</front_right_joint>
248       <back_left_joint>rear_left_wheel_joint</back_left_joint>
249       <back_right_joint>rear_right_wheel_joint</back_right_joint>
```

Lines 246–249 Which joint is which wheel.

```
250       <wheel_separation>${track + wheel_length}</wheel_separation>
```

Distance between left and right wheel centres: track + one wheel width = 0.175 m.

```
251       <wheel_base>${wheelbase}</wheel_base>
```

Distance between front and rear wheels.

```
252       <wheel_radius>${wheel_radius}</wheel_radius>
```

Wheel radius, needed to convert m/s to rad/s.

```
253       <topic>/cmd_vel</topic>
```

Listen for velocity commands on /cmd_vel. Absolute topic names (leading /) avoid Gazebo prefixing them with the model name.

```
254       <odom_topic>/odom</odom_topic>
```

Publish wheel odometry on /odom.

```
255       <tf_topic>/tf</tf_topic>
```

Publish the odom-to-base transform on /tf (inside Gazebo; not bridged).

```
256       <frame_id>odom</frame_id>
```

Odometry frame name.

```
257       <child_frame_id>base_footprint</child_frame_id>
```

Robot frame name.

```
258     </plugin>
```

End of plugin.

```
259
260     <plugin filename="gz-sim-joint-state-publisher-system" name="gz::sim::systems::JointStatePu
blisher">
```

JointStatePublisher: publishes every joint's position and velocity.

```
261       <topic>/joint_states</topic>
```

On /joint_states. This is the controller's only sensor input.

```
262     </plugin>
```

End of plugin.

```
263
264     <!-- True pose/velocity of the robot (the mecanum odometry above is computed
265          from wheel speeds and cannot see wheel slip) -->
```

Lines 264–265 Comment explaining why ground-truth odometry was added.

```
266     <plugin filename="gz-sim-odometry-publisher-system" name="gz::sim::systems::OdometryPublish
er">
```

OdometryPublisher: publishes the robot's true pose and velocity straight from the physics engine.

```
267       <odom_topic>/ground_truth/odom</odom_topic>
```

Topic name.

```
268       <odom_frame>world</odom_frame>
```

Fixed world frame.

```
269       <robot_base_frame>base_footprint</robot_base_frame>
```

Robot frame.

```
270       <odom_publish_frequency>100</odom_publish_frequency>
```

100 Hz.

```
271       <dimensions>2</dimensions>
```

Planar robot: only x, y and yaw.

```
272       <tf_topic>/ground_truth/tf</tf_topic>
```

Its transform goes on a separate topic so it does not clash with /tf.

```
273     </plugin>
```

End of plugin.

```
274   </gazebo>
```

End of the gazebo block.

```
275 </robot>
```

End of the robot.

balance_world.sdf

`src/omni_pendulum_sim/worlds/balance_world.sdf` · 39 lines

The environment the robot is spawned into: physics settings, Gazebo systems, light, ground.

```
<?xml version="1.0"?>
```

XML declaration.

```
<sdf version="1.9">
```

SDF (Simulation Description Format) is Gazebo's native file format.

```
  <world name="balance_world">
```

The world's name. It appears in Gazebo's topic names, such as `/world/balance_world/wrench/persistent`.

```
    <physics name="1ms" type="ignored">
```

Physics settings. `type="ignored"` leaves the choice of physics engine to Gazebo, whose default is DART.

```
      <max_step_size>0.001</max_step_size>
```

Each physics step advances the simulation by 1 ms.

```
      <real_time_factor>1.0</real_time_factor>
```

Try to run at real-time speed (1.0). A slow computer simply runs slower; the controller stays correct because it uses simulation time.

```
    </physics>
```

End of physics settings.

```

    <plugin filename="gz-sim-physics-system" name="gz::sim::systems::Physics"/>
```

The Physics system runs the physics engine. Without it nothing moves.

```
    <plugin filename="gz-sim-user-commands-system" name="gz::sim::systems::UserCommands"/>
```

UserCommands lets outside programs create models. `ros_gz_sim create` needs it to spawn the robot.

```
    <plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaste
r"/>
```

SceneBroadcaster sends the scene's state to the GUI so the window can draw it.

```
    <!-- Lets ROS (disturbance node) and the GUI "Apply Force Torque" panel push links -->
```

Comment.

```
    <plugin filename="gz-sim-apply-link-wrench-system" name="gz::sim::systems::ApplyLinkWrenc
h"/>
```

ApplyLinkWrench lets programs push links. The disturbance node depends on it, which is why this project has its own world instead of using `empty.sdf`.

```

    <light type="directional" name="sun">
      <cast_shadows>true</cast_shadows>
      <pose>0 0 10 0 0 0</pose>
      <diffuse>0.8 0.8 0.8 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <direction>-0.5 0.1 -0.9</direction>
    </light>
```

Lines 15–21 A sun: a directional light that casts shadows, purely for viewing.

```
22
23      <model name="ground_plane">
24        <static>true</static>
25        <link name="link">
26          <collision name="collision">
27            <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
28          </collision>
29          <visual name="visual">
30            <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
31            <material>
32              <ambient>0.8 0.8 0.8 1</ambient>
33              <diffuse>0.8 0.8 0.8 1</diffuse>
34            </material>
35          </visual>
36        </link>
37      </model>
```

Lines 23–37 The ground: a static (immovable) 100 m plane with collision, so the wheels have something to push against, and a matching grey visual.

```
38    </world>
39  </sdf>
```

Lines 38–39 Closing tags.

setup.py

`src/omni_pendulum_sim/setup.py` · 35 lines

Tells colcon what to install and where, and creates the commands that `ros2 run` starts.

```
from glob import glob
```

`glob` finds files matching a pattern, e.g. every .launch.py file.

```
import os
```

`os.path.join` for building install paths.

```

from setuptools import find_packages, setup
```

setuptools is how Python packages are built and installed; ROS 2 Python packages (`ament_python`) use it.

```

package_name = 'omni_pendulum_sim'
```

The package name, used several times below.

```

setup(
```

`setup(...)` describes how to build and install the package. `colcon build` runs it.

```
    name=package_name,
```

Package name.

```
    version='0.0.0',
```

Version.

```
    packages=find_packages(exclude=['test']),
```

Install the Python module folder `omni_pendulum_sim/` (the three nodes), skipping tests.

```
    data_files=[
```

`data_files` : non-Python files to copy into the install folder. Each entry is (destination folder, list of files).

```
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
```

A marker file that registers the package with the ROS 2 index, so `ros2 run` and `ros2 launch` can find it. Without it you get 'Package not found'.

```
        ('share/' + package_name, ['package.xml']),
```

Install package.xml (package metadata and dependencies).

```
        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
```

Install all launch files into share/omni_pendulum_sim/launch.

```
        (os.path.join('share', package_name, 'urdf'), glob('urdf/*')),
```

Install the URDF/xacro.

```
        (os.path.join('share', package_name, 'meshes', 'limo'), glob('meshes/limo/*_lite.da
e')),
```

Install only the simplified LIMO meshes (`*_lite.dae` , about 3.8 MB). The original 64 MB meshes stay in the source folder for reference.

```
        (os.path.join('share', package_name, 'worlds'), glob('worlds/*.sdf')),
```

Install the world file.

```
        (os.path.join('share', package_name, 'config'), glob('config/*')),
```

Install the Gazebo GUI layout.

```
20      ],
```

End of data_files.

```
21      install_requires=['setuptools'],
```

Python packages needed to install this one.

```
22      zip_safe=True,
```

The package can be installed as a compressed archive.

```
23      maintainer='user',
```

Maintainer name (placeholder).

```
24      maintainer_email='user@todo.todo',
```

Maintainer email (placeholder).

```
25      description='2D Inverted Pendulum Omnidirectional Robot Simulation',
```

Short description.

```
26      license='Apache-2.0',
```

License.

```
27      tests_require=['pytest'],
```

Packages needed to run tests.

```
28      entry_points={
```

Entry points: command names created by the install.

```
29          'console_scripts': [
```

`console_scripts` : each line creates an executable that `ros2 run` can start.

```
30              'balance_controller = omni_pendulum_sim.balance_controller:main',
```

`ros2 run omni_pendulum_sim balance_controller` calls `main()` in balance_controller.py.

```
31              'disturbance = omni_pendulum_sim.disturbance:main',
```

`disturbance` calls `main()` in disturbance.py.

```
32              'keyboard_nudge = omni_pendulum_sim.keyboard_nudge:main',
```

`keyboard_nudge` calls `main()` in keyboard_nudge.py.

```
33          ],
34      },
35  )
```

Lines 33–35 Closing brackets.

package.xml

`src/omni_pendulum_sim/package.xml` · 31 lines

The package manifest: its identity, and what it needs in order to run.

```
1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.
   w3.org/2001/XMLSchema"?>
```

Lines 1–2 XML declaration and the schema that validates this file.

```
3  <package format="3">
```

`package.xml` is the ROS package manifest; format 3 is the current version.

```
4    <name>omni_pendulum_sim</name>
```

Package name. It must match the folder and setup.py.

```
5    <version>0.0.0</version>
```

Version.

```
6    <description>2D inverted pendulum on a mecanum AgileX LIMO robot, simulated in Gazebo</descri
   ption>
```

One-line description.

```
7    <maintainer email="suryaganesh@todo.todo">suryaganesh</maintainer>
```

Maintainer. This is still a placeholder email; replace it with your own.

```
8    <license>Apache-2.0</license>
```

License.

```
9
10 <exec_depend>rclpy</exec_depend>
```

`rclpy` : the Python client library the nodes import.

```
11   <exec_depend>geometry_msgs</exec_depend>
12   <exec_depend>sensor_msgs</exec_depend>
13   <exec_depend>std_msgs</exec_depend>
```

Lines 11–13 The message packages for Twist and Vector3, JointState, and Int32.

```
14   <exec_depend>nav_msgs</exec_depend>
```

`nav_msgs` : the Odometry message type used by the bridge.

```
15   <exec_depend>launch</exec_depend>
16   <exec_depend>launch_ros</exec_depend>
```

Lines 15–16 The launch system and its ROS extensions.

```
17   <exec_depend>xacro</exec_depend>
```

`xacro` : expands robot.urdf.xacro at launch time.

```
18   <exec_depend>robot_state_publisher</exec_depend>
```

`robot_state_publisher` : publishes the robot description and transforms.

```
19   <exec_depend>ros_gz_sim</exec_depend>
```

`ros_gz_sim` : starts Gazebo and spawns the robot.

```
20   <exec_depend>ros_gz_bridge</exec_depend>
```

`ros_gz_bridge` : connects ROS 2 and Gazebo topics.

```
21   <exec_depend>ros_gz_interfaces</exec_depend>
```

`ros_gz_interfaces` : the EntityWrench and Entity messages used for nudges.

```
22
23    <test_depend>ament_copyright</test_depend>
24    <test_depend>ament_flake8</test_depend>
25    <test_depend>ament_pep257</test_depend>
26    <test_depend>python3-pytest</test_depend>
```

Lines 23–26 Packages needed only to run the lint tests in `test/`.

```
27
28    <export>
29      <build_type>ament_python</build_type>
30    </export>
31  </package>
```

Lines 28–31 Tells colcon this is a pure Python package (`ament_python`), and closes the file.